Linux System Programming — Hands-On Lab Manual

📋 Prerequisites

- A Linux machine (Ubuntu 20.04+ recommended) or VM
- GCC, Make, binutils installed (`sudo apt install build-essential binutils`)
- Basic C programming knowledge
- Terminal/shell familiarity

```
# Quick setup — run this first!
sudo apt update
sudo apt install -y build-essential binutils nasm hexdump \
    xxd elfutils patchelf openssl libssl-dev tree strace ltrace
```

LAB 1: GCC Compilation Stages & Programming Languages

🎯 Mission: "The Compilation Pipeline"

> ***Story:*** *You are a new recruit at a systems software company. Your first task is to understand exactly what happens when source code becomes an executable. Your mentor has left you a series of challenges to prove you understand each stage.*

📖 Background

The GCC compilation pipeline has 4 stages:

```
Source (.c) → Preprocessor → Compiler → Assembler → Linker → Executable
              (.i)           (.s)        (.o)        (a.out)
```

Stage	Flag	Input	Output	What It Does
Preprocessing	`-E`	`.c`	`.i`	Macro expansion, include insertion, conditional compilation
Compilation	`-S`	`.i`	`.s`	Converts C to assembly language
Assembly	`-c`	`.s`	`.o`	Converts assembly to machine code (object file)
Linking	(none)	`.o`	executable	Resolves symbols, combines objects, produces final binary

🏁 Exercise 1.1: Observe Each Stage

Objective: Generate and examine the output of each compilation stage.

Step 1: Navigate to the lab directory and examine the template:

```
cd code/system_programming/lab01_gcc_compilation/
cat pipeline_demo.c
```

Step 2: Run each stage separately:

```
# Stage 1: Preprocessing — expand macros, includes
gcc -E pipeline_demo.c -o pipeline_demo.i
echo "=== Lines in preprocessed file ==="
```

```
wc -l pipeline_demo.i

# Stage 2: Compilation — generate assembly
gcc -S pipeline_demo.i -o pipeline_demo.s
echo "=== Assembly output ==="
cat pipeline_demo.s

# Stage 3: Assembly — generate object file
gcc -c pipeline_demo.s -o pipeline_demo.o
echo "=== Object file type ==="
file pipeline_demo.o

# Stage 4: Linking — produce executable
gcc pipeline_demo.o -o pipeline_demo -lm
echo "=== Final executable ==="
file pipeline_demo
```

Step 3: Answer these questions in `answers.txt`:

1. How many lines does the preprocessed file `.i` have? Why so many?
2. Find the `main` label in the `.s` file. What calling convention is used?
3. What format is the `.o` file? (hint: use `file` command)
4. Run `./pipeline_demo` — what output do you see?

✅ **Checkpoint:** You should have 4 intermediate files: `.i`, `.s`, `.o`, and the executable.

🏁 Exercise 1.2: The Preprocessor Deep Dive

Objective: Master preprocessor directives and understand macro expansion.

Step 1: Examine `macro_magic.c` template.

Step 2: Compile with preprocessing only:

```
gcc -E macro_magic.c -o macro_magic.i
```

Step 3: Compare the original and preprocessed versions:

```
diff <(cat macro_magic.c) <(tail -20 macro_magic.i)
```

Step 4: Experiment with conditional compilation:

```
gcc -DPLATFORM_ARM macro_magic.c -o macro_arm
gcc -DPLATFORM_X86 macro_magic.c -o macro_x86
./macro_arm
./macro_x86
```

🎮 **Challenge:** Add a new platform `PLATFORM_RISCV` that prints "Running on RISC-V architecture" and uses a page size of 4096.

🏁 Exercise 1.3: Multi-Language Interoperability

Objective: Demonstrate how C can interface with Assembly and understand language boundaries.

Step 1: Examine the files `main_caller.c` and `asm_add.s`.

Step 2: Build the mixed-language program:

```
# Assemble the .s file
nasm -f elf64 asm_add.s -o asm_add.o    # For x86_64
# Compile the C file
gcc -c main_caller.c -o main_caller.o
# Link together
gcc main_caller.o asm_add.o -o mixed_lang
./mixed_lang
```

Step 3: Use `objdump` to see how the call crosses language boundaries:

```
objdump -d mixed_lang | grep -A 10 "call.*asm_add"
```

🎮 **Challenge:** Write an assembly function `asm_multiply` that multiplies two integers, call it from C.

LAB 2: GCC Optimization & Profile-Guided Optimization

🎯 Mission: "The Speed Demon"

> ***Story:*** *Your company's flagship product is too slow. The CTO has challenged you to squeeze every last drop of performance using compiler optimizations. You have 1 hour to show measurable speedups.*

📖 Background

GCC Optimization Levels:

Level	Flag	Description
No optimization	`-O0`	Default. Fast compile, easy debug, slow runtime
Basic optimization	`-O1`	Reduces code size and execution time
Standard optimization	`-O2`	Most recommended. Good balance
Aggressive optimization	`-O3`	Auto-vectorization, function inlining
Size optimization	`-Os`	Optimize for binary size
Extreme optimization	`-Ofast`	`-O3` + fast-math (may break IEEE compliance)

Key Optimization Techniques:

- **Dead code elimination** — removes unreachable code
- **Loop unrolling** — reduces branch overhead
- **Function inlining** — eliminates call overhead
- **Constant folding** — evaluates constant expressions at compile time
- **Vectorization** — uses SIMD instructions (SSE/AVX)

🏁 Exercise 2.1: Optimization Level Shootout

Objective: Measure the real performance impact of each optimization level.

Step 1: Navigate and examine the benchmark template:

```
cd code/system_programming/lab01_gcc_compilation/
cat optimization_benchmark.c
```

Step 2: Compile at each optimization level and measure:

```
# Compile at all levels
for opt in O0 O1 O2 O3 Os Ofast; do
    gcc -${opt} optimization_benchmark.c -o bench_${opt} -lm
    echo "=== ${opt} ==="
    ls -la bench_${opt}    # Check binary size
    time ./bench_${opt}    # Measure runtime
    echo ""
done
```

Step 3: Fill in this results table in `answers.txt`:

Level	Binary Size (bytes)	Real Time (s)	Speedup vs -O0
-O0			1.0x
-O1			
-O2			
-O3			
-Os			
-Ofast			

Step 4: Examine the assembly differences:

```
gcc -O0 -S optimization_benchmark.c -o bench_O0.s
gcc -O3 -S optimization_benchmark.c -o bench_O3.s
diff bench_O0.s bench_O3.s | head -80
```

🎮 **Challenge:** Find a case where `-O3` produces DIFFERENT output than `-O0` (hint: floating-point operations with `-Ofast`).

🏁 Exercise 2.2: Specific Optimization Flags

Objective: Understand individual optimization flags that compose the `-O` levels.

Step 1: See what flags each level enables:

```
# List all optimizations at each level
gcc -Q -O0 --help=optimizers | grep enabled
gcc -Q -O2 --help=optimizers | grep enabled
gcc -Q -O3 --help=optimizers | grep enabled
```

Step 2: Try specific optimizations on `specific_opts.c`:

```
# No optimization baseline
gcc -O0 specific_opts.c -o spec_base
objdump -d spec_base | grep -A 30 "<dead_code_test>"

# Dead code elimination only
gcc -O0 -fdce specific_opts.c -o spec_dce
objdump -d spec_dce | grep -A 30 "<dead_code_test>"
```

```
# Loop unrolling
gcc -O0 -funroll-loops specific_opts.c -o spec_unroll
objdump -d spec_unroll | grep -A 50 "<loop_test>"

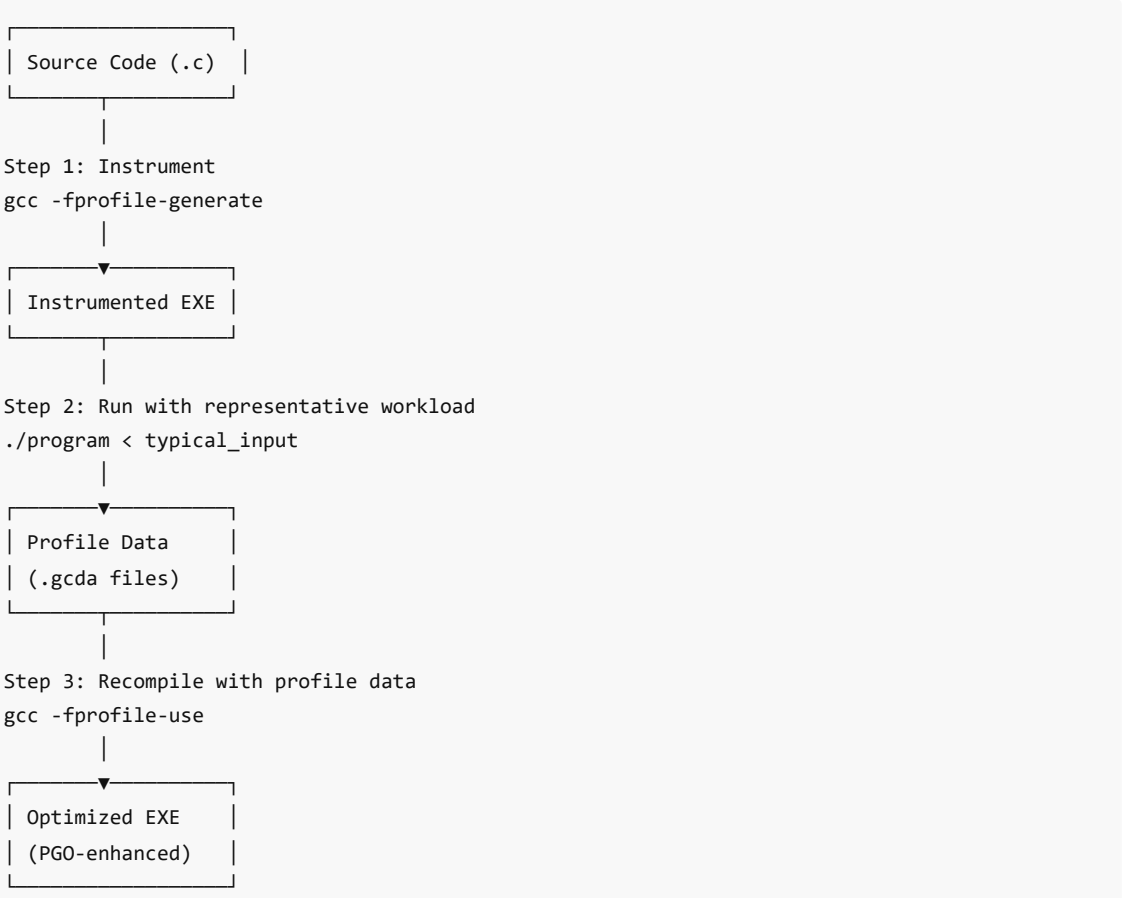
# Function inlining
gcc -O0 -finline-functions specific_opts.c -o spec_inline
objdump -d spec_inline | grep -c "call.*tiny_function"
```

Step 3: Document which optimization eliminated what code.

🏁 Exercise 2.3: Profile-Guided Optimization (PGO)

Objective: Use real runtime profiling data to guide the compiler's optimization decisions.

PGO Workflow:



Step 1: Examine `pgo_demo.c` template.

Step 2: Create instrumented build:

```
gcc -O2 -fprofile-generate pgo_demo.c -o pgo_instrument -lm
```

Step 3: Run with representative workloads to generate profile data:

```
./pgo_instrument         # Generates .gcda files
ls *.gcda                # Verify profile data was created
```

Step 4: Rebuild using profile data:

```
gcc -O2 -fprofile-use -fprofile-correction pgo_demo.c -o pgo_optimized -lm
```

Step 5: Compare performance:

```
# Also build a plain O2 version for comparison
gcc -O2 pgo_demo.c -o pgo_plain -lm

echo "=== Plain -O2 ==="
time ./pgo_plain

echo "=== PGO Optimized ==="
time ./pgo_optimized
```

Step 6: Record the results:

Build	Time (s)	Improvement
-O2 plain		baseline
-O2 + PGO		

🎮 **Challenge:** Modify `pgo_demo.c` to have a heavily branched function where one branch is taken 95% of the time. Show that PGO improves branch prediction by examining `gcov` data:

```
gcc -O2 -fprofile-generate --coverage pgo_demo.c -o pgo_cov -lm
./pgo_cov
gcov pgo_demo.c
cat pgo_demo.c.gcov
```

LAB 3: Binutils & ELF Analysis

🎯 Mission: "The Binary Archaeologist"

> ***Story:*** *An ancient binary artifact has been discovered in a legacy system. Your team needs to reverse-engineer its structure using the binutils toolkit. Each tool reveals a different layer of the binary.*

📖 Background

The GNU Binutils Toolkit:

Tool	Purpose
`objdump`	Disassemble, display headers, sections, symbols
`readelf`	ELF-specific detailed information
`nm`	List symbols from object files
`strings`	Extract printable strings
`strip`	Remove symbols
`objcopy`	Copy/transform object files
`addr2line`	Convert addresses to file:line

`size`	Section sizes
`ar`	Archive management (static libraries)
`ld`	The GNU linker

ELF (Executable and Linkable Format) Structure:

```
┌─────────────────────────────────┐
│       ELF Header                │  ← Magic number, architecture, entry point
├─────────────────────────────────┤
│   Program Header Table          │  ← Segments (for runtime loading)
├─────────────────────────────────┤
│   .text  (code)                 │  ← Executable instructions
├─────────────────────────────────┤
│   .rodata (read-only data)      │  ← String literals, constants
├─────────────────────────────────┤
│   .data  (initialized data)     │  ← Global/static initialized vars
├─────────────────────────────────┤
│   .bss   (uninitialized)        │  ← Global/static uninitialized vars
├─────────────────────────────────┤
│   .symtab (symbol table)        │  ← Function/variable names
├─────────────────────────────────┤
│   .strtab (string table)        │  ← Symbol name strings
├─────────────────────────────────┤
│   .rel.text (relocations)       │  ← Relocation entries
├─────────────────────────────────┤
│   Section Header Table          │  ← Section metadata
└─────────────────────────────────┘
```

🏁 Exercise 3.1: The ELF Autopsy

Objective: Perform a complete analysis of an ELF binary using binutils.

Step 1: Navigate and build the artifact:

```
cd code/system_programming/lab02_binutils_elf/
gcc -g elf_artifact.c -o elf_artifact
```

Step 2: Examine the ELF header:

```
readelf -h elf_artifact
```

Record: What is the entry point address? What machine type? 32 or 64-bit?

Step 3: Examine sections:

```
readelf -S elf_artifact
echo "=== Section Sizes ==="
size elf_artifact
```

Step 4: List symbols:

```
nm elf_artifact
nm elf_artifact | grep " T "    # Text (code) symbols
```

```
nm elf_artifact | grep " D "    # Data symbols
nm elf_artifact | grep " B "    # BSS symbols
```

Step 5: Extract strings:

```
strings elf_artifact | head -30
```

Step 6: Disassemble main:

```
objdump -d elf_artifact | grep -A 40 "<main>"
```

Step 7: Examine program headers (segments):

```
readelf -l elf_artifact
```

Step 8: Fill in the ELF analysis report in `answers.txt` : ``` ELF Analysis Report

Entry Point: 0x_____ Architecture: _____ Type: _____ .text size: _____ bytes .data size: _____ bytes .bss size: _____ bytes

# of symbols: _____

# of sections: _____

Hidden message: _____ (find it with strings!)

```
---

### 🏁 Exercise 3.2: Symbol Surgery

**Objective:** Manipulate binary symbols using strip, objcopy, and patchelf.

**Step 1:** Check original symbol count:
```bash
nm elf_artifact | wc -l
ls -la elf_artifact
```

Step 2: Strip debug symbols:

```bash
cp elf_artifact elf_stripped
strip elf_stripped
echo "=== After stripping ==="
nm elf_stripped 2>&1 | head -5
ls -la elf_stripped
# Compare sizes
echo "Original: $(stat -c %s elf_artifact) bytes"
echo "Stripped: $(stat -c %s elf_stripped) bytes"
```
```

Step 3: Add a custom section:

```
echo "CLASSIFIED: Property of Lab Team Alpha" > secret.txt
objcopy --add-section .secret=secret.txt elf_artifact elf_with_secret
readelf -S elf_with_secret | grep secret
objcopy --dump-section .secret=/dev/stdout elf_with_secret
```

Step 4: Convert address to source line:

```
# Get the address of main
MAIN_ADDR=$(nm elf_artifact | grep " T main" | awk '{print $1}')
addr2line -e elf_artifact 0x${MAIN_ADDR}
```

🎮 **Challenge:** Extract the `.text` section into a raw binary file and verify it matches the disassembly:

```
objcopy -O binary --only-section=.text elf_artifact text_raw.bin
xxd text_raw.bin | head -20
```

🏁 Exercise 3.3: The Linker Map Challenge

Objective: Understand the linking process by examining linker maps and object file composition.

Step 1: Compile with map file generation:

```
gcc -g -Wl,-Map=link_map.txt elf_artifact.c -o elf_artifact
cat link_map.txt | head -80
```

Step 2: Build a multi-file program to understand symbol resolution:

```
gcc -c elf_artifact.c -o elf_artifact.o
# Look at relocations in the object file
readelf -r elf_artifact.o
```

Step 3: Understand dynamic linking:

```
# Show dynamic dependencies
ldd elf_artifact
# Show dynamic symbol table
readelf -d elf_artifact
```

🎮 **Challenge:** Create a custom linker script that places a custom section `.mydata` at a specific address. Use `elf_artifact.c` with a variable attributed to `.mydata` :

```
__attribute__((section(".mydata"))) int special_var = 0xDEADBEEF;
```

LAB 4: Introduction to Linux & Filesystem Hierarchy

🎯 Mission: "The Filesystem Explorer"

> ***Story:*** *You've just been dropped into a Linux system with no documentation. You must map out the entire filesystem, understand what lives where, and prove your knowledge by navigating like a pro.*

📖 Background

Linux Filesystem Hierarchy Standard (FHS):

```
/
├── bin/       → Essential command binaries (ls, cp, cat)
├── boot/      → Boot loader files, kernel image
├── dev/       → Device files (tty, null, zero, random)
├── etc/       → System-wide configuration files
├── home/      → User home directories
├── lib/       → Essential shared libraries
├── media/     → Mount points for removable media
├── mnt/       → Temporary mount points
├── opt/       → Optional/third-party software
├── proc/      → Virtual filesystem for process/kernel info
├── root/      → Root user's home directory
├── run/       → Runtime variable data
├── sbin/      → System administration binaries
├── srv/       → Data for services (web, FTP)
├── sys/       → Virtual filesystem for kernel/device info
├── tmp/       → Temporary files
├── usr/       → Secondary hierarchy (usr/bin, usr/lib, usr/share)
└── var/       → Variable data (logs, spool, cache)
```

🏁 Exercise 4.1: Filesystem Scavenger Hunt

Objective: Find specific items hidden throughout the filesystem. Answer each question.

Complete these challenges by writing the command and answer:

#	Challenge	Command	Answer
1	Find the kernel version file	`cat /proc/version`	
2	How many CPUs does the system have?	`nproc` or `cat /proc/cpuinfo | grep processor | wc -l`	
3	What is the hostname?	`cat /etc/hostname`	
4	Find total RAM in KB	`cat /proc/meminfo | grep MemTotal`	
5	List all block devices	`lsblk`	
6	Find the system's default shell	`cat /etc/passwd | grep root`	
7	What filesystem type is `/` mounted as?	`df -T /`	
8	Find the system uptime	`cat /proc/uptime`	
9	List all environment variables	`env`	
10	Find the GCC version installed	`gcc --version`	

🎮 Bonus (10 pts each):

- Find a hidden file in `/etc/` that contains the system's release information
- Determine the PID of the init process and what init system is running
- Find all SUID binaries on the system: `find / -perm -4000 -type f 2>/dev/null`

🏁 Exercise 4.2: The /proc and /sys Virtual Filesystems

Objective: Explore the virtual filesystems that expose kernel internals.

Step 1: Explore `/proc` for process information:

```
# Current process info
echo "My PID: $$"
ls /proc/$$/
cat /proc/$$/status | head -20
cat /proc/$$/maps | head -10
cat /proc/$$/cmdline

# System-wide info
cat /proc/loadavg
cat /proc/stat | head -5
cat /proc/interrupts | head -20
```

Step 2: Explore `/sys` for device/kernel information:

```
# CPU info
ls /sys/devices/system/cpu/
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq 2>/dev/null

# Block devices
ls /sys/block/
cat /sys/block/sda/size 2>/dev/null

# Kernel parameters
ls /sys/kernel/
cat /sys/kernel/hostname
```

Step 3: Write a script `sysinfo_collector.sh` that outputs a formatted system report: ``` ========== SYSTEM INFO REPORT ========== Hostname: _____ Kernel: _____ Architecture: _____ CPUs: _____ Total RAM: _____ MB Uptime: _____ seconds Load Average: _____ Root FS Type: _____

```
---

# LAB 5: Encryption Techniques

## 🎯 Mission: "The Crypto Challenge"

> **Story:** Your team needs to implement secure communication for a field deployment. You must demonstrate proficiency with symmetric encryption, hashing, and asymmetric encryption using both command-line tools and C
```

````
programming.

### 📖 Background

**Encryption Types Overview:**

| Type | Examples | Key Property | Use Case |
|------|----------|-------------|----------|
| **Symmetric** | AES, DES, ChaCha20 | Same key for encrypt/decrypt | Bulk data encryption |
| **Asymmetric** | RSA, ECC | Public/Private key pair | Key exchange, digital signatures |
| **Hashing** | SHA-256, MD5 | One-way, no key | Integrity verification |
| **HMAC** | HMAC-SHA256 | Hash + secret key | Message authentication |

### 🏁 Exercise 5.1: Command-Line Cryptography

**Objective:** Use OpenSSL to perform various encryption operations.

**Step 1:** Symmetric Encryption (AES):
```bash
cd code/system_programming/lab15_encryption/

# Create a secret message
echo "TOP SECRET: The launch code is 12345" > plaintext.txt

# Encrypt with AES-256-CBC
openssl enc -aes-256-cbc -salt -pbkdf2 -in plaintext.txt -out encrypted.bin -pass pass:MySecretKey123

# View the encrypted file (it's binary)
xxd encrypted.bin | head -5

# Decrypt
openssl enc -d -aes-256-cbc -pbkdf2 -in encrypted.bin -out decrypted.txt -pass pass:MySecretKey123
cat decrypted.txt

# Verify they match
diff plaintext.txt decrypted.txt && echo "SUCCESS: Files match!"
````

Step 2: Hashing:

```
# Generate hashes
echo -n "Hello, Linux!" | md5sum
echo -n "Hello, Linux!" | sha1sum
echo -n "Hello, Linux!" | sha256sum
echo -n "Hello, Linux!" | sha512sum

# File integrity check
sha256sum encrypted.bin > checksum.sha256
sha256sum -c checksum.sha256
```

Step 3: Asymmetric Encryption (RSA):

```
# Generate RSA key pair
openssl genrsa -out private_key.pem 2048
openssl rsa -in private_key.pem -pubout -out public_key.pem

# Encrypt with public key
```
```

```
openssl rsautl -encrypt -inkey public_key.pem -pubin -in plaintext.txt -out rsa_encrypted.bin

# Decrypt with private key
openssl rsautl -decrypt -inkey private_key.pem -in rsa_encrypted.bin -out rsa_decrypted.txt
cat rsa_decrypted.txt
```

Step 4: Digital Signatures:

```
# Sign a file
openssl dgst -sha256 -sign private_key.pem -out signature.bin plaintext.txt

# Verify the signature
openssl dgst -sha256 -verify public_key.pem -signature signature.bin plaintext.txt
```

🏁 Exercise 5.2: Encryption in C

Objective: Implement encryption operations programmatically using OpenSSL's libcrypto.

Step 1: Examine and build `crypto_demo.c`:

```
gcc crypto_demo.c -o crypto_demo -lssl -lcrypto
./crypto_demo
```

Step 2: The template demonstrates:

- SHA-256 hashing
- AES-256-CBC encryption/decryption
- HMAC generation

Step 3: Extend the program to:

1. Read a file from command line argument
2. Compute its SHA-256 hash
3. Encrypt it with a password provided by the user
4. Write the encrypted output to a `.enc` file
5. Verify by decrypting back

🎮 **Challenge — The Crypto Relay Race:**

1. Partner A creates a message and encrypts it with AES
2. Partner A generates an RSA key pair and encrypts the AES key with RSA public key
3. Partner A sends: encrypted message + encrypted AES key + RSA public key
4. Partner B decrypts the AES key using RSA private key (wait — this won't work!)
5. Discuss: Why doesn't this work? How should keys be exchanged? (Answer: Partner B should generate the RSA keys and share their public key with A)

LAB 6: Linux Binary Analysis — Tools

🎯 Mission: "The Binary Forensics Lab"

> ***Story:*** *A suspicious binary has been found on a production server. You are the forensics expert called in to analyze it without executing it. Use every tool in your arsenal to determine what it does.*

📖 Background

Binary Analysis Toolkit:

Tool	Purpose	Example
`file`	Identify file type	`file mystery`
`strings`	Extract text strings	`strings mystery`
`hexdump/xxd`	Hex viewer	`xxd mystery | head`
`readelf`	ELF structure analysis	`readelf -a mystery`
`objdump`	Disassembly	`objdump -d mystery`
`nm`	Symbol listing	`nm mystery`
`ldd`	Shared library deps	`ldd mystery`
`strace`	System call tracing	`strace ./mystery`
`ltrace`	Library call tracing	`ltrace ./mystery`
`size`	Section sizes	`size mystery`

🏁 Exercise 6.1: The Mystery Binary Challenge

Objective: Analyze `mystery_binary.c` (compiled by your instructor — pretend you don't have the source).

Step 1: Build the mystery binary (instructor preparation step):

```
cd code/system_programming/lab02_binutils_elf/
gcc -O2 mystery_binary.c -o mystery -lm
strip mystery    # Remove symbols to make it harder!
```

Step 2: Begin your analysis — DO NOT look at the source code!

Phase 1 — Basic Identification:

```
file mystery
size mystery
```

Phase 2 — String Analysis:

```
strings mystery | sort -u
strings mystery | grep -i "password\|secret\|key\|flag"
```

Phase 3 — Library Dependencies:

```
ldd mystery
```

Phase 4 — Disassembly:

```
objdump -d mystery | head -100
objdump -d mystery | grep "call" | head -20
```

Phase 5 — Runtime Tracing (safe to run in this case):

```
strace ./mystery 2>&1 | head -40
ltrace ./mystery 2>&1 | head -40
```

Step 3: Write a forensics report: ``` Binary Forensics Report

File Type: _____ Architecture: _____ Linked Libraries: _____ Interesting Strings: _____ System Calls Made: _____ Library Calls Made: _____ Probable Function: _____ Risk Assessment: LOW / MEDIUM / HIGH

```
---

### 🏁 Exercise 6.2: Build Your Own Analysis Script

**Objective:** Create an automated binary analysis tool.

**Step 1:** Examine and complete `binary_analyzer.sh`:

The template provides a skeleton. You need to fill in the analysis functions to create a comprehensive automated report generator.

**Step 2:** Test it:
```bash
chmod +x binary_analyzer.sh
./binary_analyzer.sh mystery
./binary_analyzer.sh /usr/bin/ls
```

🎮 **Challenge:** Extend the analyzer to:

1. Check if the binary is packed/compressed (hint: look for UPX signatures)
2. Extract and display all URLs/IPs found in strings
3. Identify potentially dangerous system calls (execve, ptrace, socket)

MINI-PROJECT: "The Secure Build Pipeline"

Overview

Build an end-to-end secure compilation and analysis pipeline that:

1. **Compiles** a C project through all GCC stages with verbose output
2. **Optimizes** using PGO based on test workloads
3. **Encrypts** the final binary with AES-256
4. **Signs** it with an RSA key
5. **Analyzes** and generates a full binary report
6. **Verifies** integrity before deployment

Project Structure

```
project_secure_pipeline/
├── Makefile
├── src/
│   └── app.c              ← Your application
├── keys/
│   ├── private_key.pem    ← Generated RSA private key
```
```

```
│   └── public_key.pem     ← Generated RSA public key
├── build/
│   ├── app.i              ← Preprocessed
│   ├── app.s              ← Assembly
│   ├── app.o              ← Object
│   ├── app                ← Executable
│   ├── app.enc            ← Encrypted binary
│   ├── app.sig            ← Digital signature
│   └── app.sha256         ← SHA-256 checksum
├── profile_data/          ← PGO data
└── reports/
    └── analysis_report.txt ← Binary analysis report
```

Requirements

Phase 1: The Build

- Write a Makefile that compiles through each stage with verbose output
- Include targets: `preprocess`, `compile`, `assemble`, `link`, `all`

Phase 2: Optimization

- Add PGO targets: `pgo-instrument`, `pgo-train`, `pgo-optimize`
- Compare PGO vs non-PGO builds and log results

Phase 3: Security

- Generate RSA-2048 key pair
- Encrypt the binary with AES-256-CBC
- Sign the binary with RSA
- Create SHA-256 checksum

Phase 4: Analysis

- Generate comprehensive binary analysis report
- Include: ELF headers, section info, symbols, strings, dependencies
- Add a `verify` target that checks signature and checksum

Deliverables

1. Complete working Makefile
2. Source code (`app.c`)
3. Generated reports
4. Screenshot/log showing full pipeline execution

PART 2: ELF, Libraries, Debugging & Process Management

LAB 7: ELF Analysis — Deep Dive

🎯 Mission: "The ELF Surgeon"

> ***Story:*** *A critical production binary is crashing intermittently. You have the ELF binary and limited debug info. You must perform deep ELF analysis to understand the binary's structure, identify its sections, and trace its dependencies — all without source code access.*

📖 Background

ELF File Types:

Type	Description	Example
`ET_REL`	Relocatable (object file)	`.o` files from `gcc -c`
`ET_EXEC`	Executable (fixed address)	Statically linked binaries
`ET_DYN`	Shared object / PIE executable	`.so` files, modern executables
`ET_CORE`	Core dump	Crash dumps

Key ELF Sections for Analysis:

Section	Purpose
`.text`	Executable code
`.plt` / `.got`	Procedure Linkage Table / Global Offset Table (dynamic linking)
`.init` / `.fini`	Constructor / destructor code
`.dynamic`	Dynamic linking information
`.dynsym`	Dynamic symbol table
`.rel.plt`	Relocations for PLT entries
`.note`	Vendor/build information
`.eh_frame`	Exception handling / stack unwinding

🏁 Exercise 7.1: ELF Section Deep Analysis

Objective: Map every section of an ELF binary and understand its role.

Step 1: Build the analysis target:

```
cd code/system_programming/lab02_binutils_elf/
gcc -g -O2 elf_artifact.c -o elf_deep -lm
```

Step 2: Complete ELF section map:

```
# Full section listing with flags
readelf -S -W elf_deep

# Identify which sections are:
#   A = Allocatable (loaded into memory)
#   X = Executable
#   W = Writable
readelf -S elf_deep | grep -E "AX|WA|A " | awk '{print $2, $7}'
```

Step 3: Examine the PLT/GOT (dynamic linking machinery):

```
# PLT entries — stubs that jump to shared library functions
objdump -d -j .plt elf_deep

# GOT entries — addresses resolved at runtime
```

```
readelf -r elf_deep | head -20

# Show dynamic symbols
readelf --dyn-syms elf_deep
```

Step 4: Examine constructor/destructor sections:

```
objdump -d -j .init elf_deep
objdump -d -j .fini elf_deep

# .init_array / .fini_array function pointers
readelf -x .init_array elf_deep 2>/dev/null
```

Step 5: Fill in the analysis worksheet in `answers.txt`: ``` ELF Deep Analysis Worksheet

Total sections: ___ Loadable segments: ___ PLT entries (count): ___ GOT entries (count): ___ Has .init_array: YES / NO Has .note section: YES / NO Has DWARF debug info: YES / NO Stack executable: YES / NO (check GNU_STACK) PIE enabled: YES / NO (check Type: DYN) RELRO: NONE / PARTIAL / FULL

```
**🎮 Challenge:** Write a C program that registers functions in `.init_array` and `.fini_array` using
`__attribute__((constructor))` and `__attribute__((destructor))`. Verify they execute before/after `main()`.

---

### 🏁 Exercise 7.2: ELF Patching

**Objective:** Modify an ELF binary without recompiling.

**Step 1:** Build the patchable target:
```bash
gcc -g elf_patch_target.c -o patch_me
./patch_me     # Observe original behavior
```

Step 2: Find the string to patch:

```
strings -t x patch_me | grep "Access Denied"
# Note the hex offset
```

Step 3: Patch the string in-place:

```
# Replace "Access Denied" with "Access Granted" (same length!)
cp patch_me patch_me_modified
printf 'Access Granted' | dd of=patch_me_modified bs=1 seek=$((0xOFFSET)) conv=notrunc
./patch_me_modified
```

Step 4: Use `objcopy` to replace an entire section:

```
echo "Patched by $(whoami) on $(date)" > patch_note.txt
objcopy --add-section .patch_log=patch_note.txt patch_me patch_me_noted
```
```

```
readelf -S patch_me_noted | grep patch_log
```

🎮 **Challenge:** Patch the binary to skip an authentication check by replacing a conditional jump (`je` / `jne`) with NOPs (`0x90`). Use `objdump -d` to find the branch instruction.

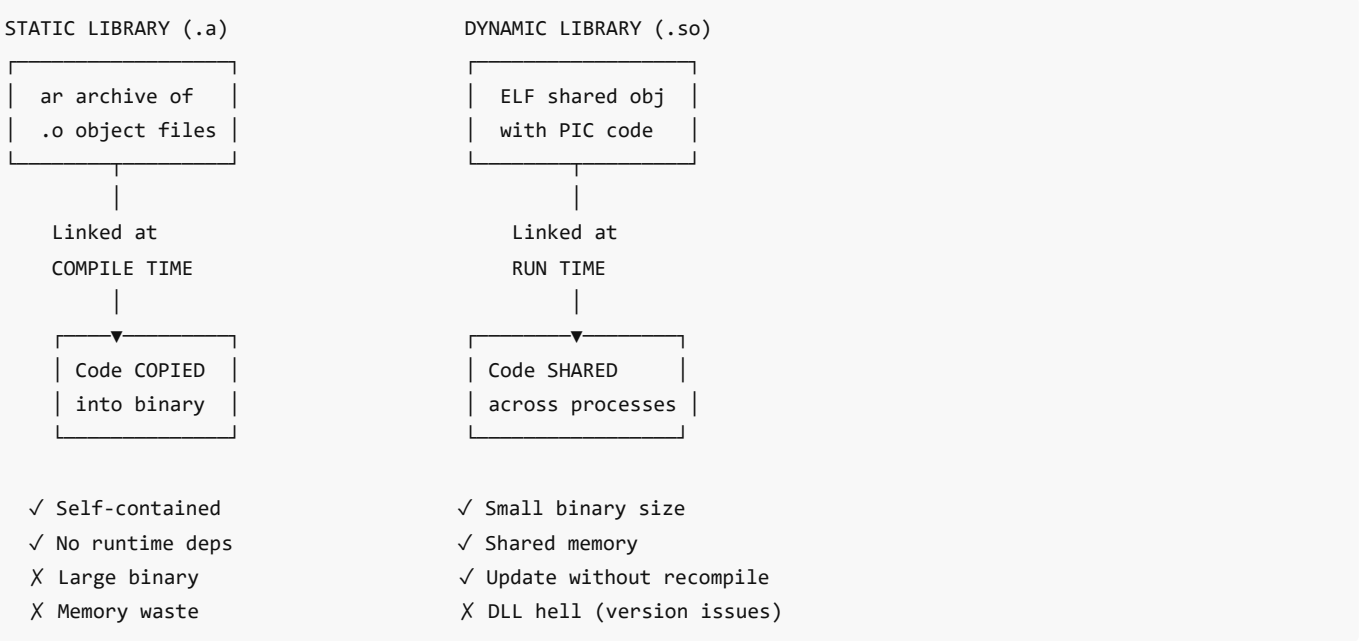
LAB 8: Static & Dynamic Libraries

🎯 Mission: "The Library Architect"

> ***Story:*** *Your team is building a math utilities toolkit. You need to package it as both a static library (for embedded deployment) and a shared library (for desktop applications). Understanding the tradeoffs is critical.*

📖 Background

Static vs Dynamic Libraries:



🏁 Exercise 8.1: Building a Static Library

Objective: Create, inspect, and link a static library.

Step 1: Navigate and examine the library source:

```
cd code/system_programming/lab03_libraries/
cat mathlib.h
cat mathlib_basic.c
cat mathlib_advanced.c
```

Step 2: Compile to object files:

```
gcc -c -Wall mathlib_basic.c -o mathlib_basic.o
gcc -c -Wall mathlib_advanced.c -o mathlib_advanced.o -lm
```

Step 3: Create the static library archive:

```
ar rcs libmathutil.a mathlib_basic.o mathlib_advanced.o

# Inspect the archive
ar t libmathutil.a          # List contents
nm libmathutil.a            # Show symbols
```

Step 4: Link a program against the static library:

```
gcc test_mathlib.c -L. -lmathutil -o test_static -lm

# Verify it's self-contained
ldd test_static             # Should NOT show libmathutil
file test_static
./test_static
```

Step 5: Measure the binary:

```
ls -la test_static
nm test_static | grep " T " | head -20  # Our symbols are embedded
```

🏁 Exercise 8.2: Building a Dynamic Library

Objective: Create, install, and use a shared library.

Step 1: Compile with Position Independent Code (PIC):

```
gcc -c -Wall -fPIC mathlib_basic.c -o mathlib_basic_pic.o
gcc -c -Wall -fPIC mathlib_advanced.c -o mathlib_advanced_pic.o
```

Step 2: Create the shared library:

```
gcc -shared -Wl,-soname,libmathutil.so.1 \
    -o libmathutil.so.1.0.0 \
    mathlib_basic_pic.o mathlib_advanced_pic.o -lm

# Create symlinks (standard convention)
ln -sf libmathutil.so.1.0.0 libmathutil.so.1    # SONAME link
ln -sf libmathutil.so.1 libmathutil.so           # Development link

ls -la libmathutil*
```

Step 3: Link a program against the shared library:

```
gcc test_mathlib.c -L. -lmathutil -o test_dynamic -lm

# Check dependencies
ldd test_dynamic         # Should show libmathutil.so

# Run with library path
LD_LIBRARY_PATH=. ./test_dynamic
```

Step 4: Compare static vs dynamic:

```
ls -la test_static test_dynamic
echo "Static: $(stat -c %s test_static) bytes"
echo "Dynamic: $(stat -c %s test_dynamic) bytes"
echo "Shared lib: $(stat -c %s libmathutil.so.1.0.0) bytes"
```

Step 5: Examine the shared library:

```
readelf -d libmathutil.so.1.0.0    # Dynamic section
nm -D libmathutil.so.1.0.0         # Exported symbols
objdump -T libmathutil.so.1.0.0    # Dynamic symbol table
```

🏁 Exercise 8.3: Dynamic Loading with dlopen()

Objective: Load shared libraries at runtime using the `dl` API.

Step 1: Examine and build `dlopen_demo.c`:

```
gcc dlopen_demo.c -o dlopen_demo -ldl
```

Step 2: Run it:

```
LD_LIBRARY_PATH=. ./dlopen_demo
```

Step 3: Examine how `dlopen` resolves symbols:

```
# Trace library loading
LD_DEBUG=libs LD_LIBRARY_PATH=. ./dlopen_demo 2>&1 | head -30

# Trace symbol resolution
LD_DEBUG=symbols LD_LIBRARY_PATH=. ./dlopen_demo 2>&1 | head -30
```

🎮 **Challenge:** Create a plugin system — write two shared libraries (`plugin_a.so` , `plugin_b.so`) each implementing a `plugin_run()` function. Write a host program that loads the plugin specified on the command line.

LAB 9: Error Handling & Asserts

🎯 Mission: "The Defensive Programmer"

> ***Story:*** *A teammate's code has been crashing in production with cryptic errors. You've been asked to retrofit proper error handling throughout the codebase and add defensive assertions for development builds.*

📖 Background

Linux Error Handling Mechanisms:

Mechanism	Purpose	Header
`errno`	System/library call error codes	`<errno.h>`
`perror()`	Print errno description	`<stdio.h>`
`strerror()`	Get errno string	`<string.h>`

`assert()`	Debug-time invariant check	`<assert.h>`
`static_assert()`	Compile-time check (C11)	`<assert.h>`
Return codes	Function-specific error returns	—
`setjmp/longjmp`	Non-local jumps (C "exceptions")	`<setjmp.h>`

🏁 Exercise 9.1: errno and System Call Error Handling

Objective: Master errno-based error detection and reporting for all system calls.

Step 1: Examine and build `error_handling_demo.c`:

```
cd code/system_programming/lab04_error_handling/
gcc -Wall -g error_handling_demo.c -o error_demo
./error_demo
```

Step 2: Trigger and handle various errors:

```
# The demo attempts operations that fail:
# - Opening a non-existent file (ENOENT)
# - Writing to a read-only file (EACCES)
# - Allocating impossibly large memory (ENOMEM)
# - Connecting to invalid socket (ECONNREFUSED)
```

Step 3: Examine errno values:

```
# List all errno values on your system
python3 -c "import errno; [print(f'{v:3d}: {k}') for k,v in sorted(errno.errorcode.items())]" 2>/dev/null || \
cpp -dM /usr/include/errno.h | grep "define E" | sort -t' ' -k3 -n | head -40
```

🏁 Exercise 9.2: Assertions and Defensive Programming

Objective: Use compile-time and runtime assertions effectively.

Step 1: Examine and build `assert_demo.c`:

```
# Debug build — assertions ENABLED
gcc -Wall -g -DDEBUG assert_demo.c -o assert_debug
./assert_debug

# Release build — assertions DISABLED
gcc -Wall -O2 -DNDEBUG assert_demo.c -o assert_release
./assert_release
```

Step 2: Observe assertion failure:

```
./assert_debug trigger_assert   # Pass argument to trigger assertion
# Observe the assertion message format:
#   file:line: function: Assertion 'condition' failed.
```

Step 3: Custom assertion macros: The template includes `ASSERT_MSG()`, `ASSERT_NOT_NULL()`, `PRECONDITION()`, and `POSTCONDITION()`. Test each and observe the output.

🎮 **Challenge:** Implement a `setjmp` / `longjmp` based try-catch error handler:

```
// Goal: make this syntax work in C
TRY {
    risky_operation();
} CATCH(ERR_FILE_NOT_FOUND) {
    printf("File not found!\n");
} CATCH(ERR_PERMISSION) {
    printf("Permission denied!\n");
} END_TRY;
```

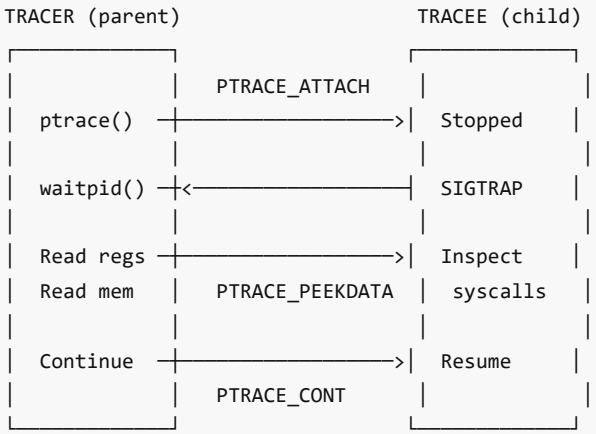
LAB 10: Linux Process Tracing — Ptrace

🎯 Mission: "The System Call Inspector"

> ***Story:*** *You're building a security monitoring tool that needs to intercept and log every system call made by a target process. The* `ptrace` *system call is your secret weapon — the same mechanism used by* `strace` *and* `gdb` *.*

📖 Background

ptrace() Overview:



Key ptrace Requests:

Request	Purpose
`PTRACE_TRACEME`	Child requests tracing by parent
`PTRACE_ATTACH`	Attach to running process
`PTRACE_PEEKTEXT`	Read word from tracee's memory
`PTRACE_POKETEXT`	Write word to tracee's memory
`PTRACE_GETREGS`	Read tracee's registers
`PTRACE_SETREGS`	Modify tracee's registers
`PTRACE_SYSCALL`	Continue, stop at next syscall entry/exit
`PTRACE_SINGLESTEP`	Execute one instruction

`PTRACE_CONT`	Continue execution
`PTRACE_DETACH`	Detach from tracee

🏁 Exercise 10.1: Build a Mini-strace

Objective: Create a tool that traces system calls made by a child process.

Step 1: Examine and build `mini_strace.c` :

```
cd code/system_programming/lab05_ptrace_antidebug/
gcc -Wall -g mini_strace.c -o mini_strace
```

Step 2: Trace a simple command:

```
./mini_strace /bin/ls
./mini_strace /bin/echo "Hello ptrace"
```

Step 3: Compare with real strace:

```
strace -c /bin/ls 2>&1 | tail -20
./mini_strace /bin/ls 2>&1 | wc -l
```

Step 4: Extend mini_strace to decode syscall names. The template includes a syscall number-to-name mapping table for x86_64.

🏁 Exercise 10.2: Memory Inspector with Ptrace

Objective: Read and modify another process's memory using ptrace.

Step 1: Examine and build `ptrace_memspy.c` :

```
gcc -Wall -g ptrace_memspy.c -o memspy
gcc -Wall -g target_program.c -o target_prog
```

Step 2: In terminal 1, run the target:

```
./target_prog
# It will print its PID and a secret value's address
```

Step 3: In terminal 2, attach and read memory:

```
./memspy <PID> <ADDRESS>
```

🎮 **Challenge:** Extend `memspy` to not just read but **modify** the target's variable at runtime using `PTRACE_POKEDATA` .

LAB 11: Linux Anti-Debugging

🎯 Mission: "The Fortress Builder"

> ***Story:*** *Your software handles sensitive cryptographic keys. You need to implement anti-debugging protections to make reverse engineering harder. Then, as a red team exercise, try to bypass your own protections.*

📖 Background

Common Anti-Debugging Techniques:

Technique	How It Works
`ptrace(PTRACE_TRACEME)`	A process can only be traced once — if it traces itself, debuggers can't attach
`/proc/self/status` check	`TracerPid` field is non-zero when being debugged
Timing checks	Breakpoints cause execution delays detectable by `RDTSC` or `clock_gettime`
Signal tricks	Debuggers intercept signals differently than normal execution
INT3 detection	Breakpoint instruction (`0xCC`) can be detected in own code
`/proc/self/maps`	Check for injected libraries or unusual memory mappings

🏁 Exercise 11.1: Implementing Anti-Debug Protections

Objective: Build a program with multiple anti-debugging layers.

Step 1: Examine and build `anti_debug.c`:

```
cd code/system_programming/lab05_ptrace_antidebug/
gcc -Wall -g anti_debug.c -o anti_debug
```

Step 2: Test without debugger:

```
./anti_debug            # Should run normally
echo $?                 # Exit code 0 = success
```

Step 3: Test with debugger:

```
gdb ./anti_debug        # Anti-debug should trigger
strace ./anti_debug     # Ptrace-based detection fires
```

Step 4: Study each protection layer in the source code and document how each one detects debugging.

🏁 Exercise 11.2: Bypassing Anti-Debug (Red Team)

Objective: Defeat each anti-debugging technique.

Step 1: Bypass ptrace self-trace:

```
# Method 1: LD_PRELOAD a fake ptrace
gcc -shared -fPIC -o fake_ptrace.so fake_ptrace.c
LD_PRELOAD=./fake_ptrace.so ./anti_debug

# Method 2: Patch the binary (NOP out the ptrace call)
```

Step 2: Bypass /proc/self/status check:

```
# Use a custom /proc via mount namespace or LD_PRELOAD fopen
```

Step 3: Document each bypass technique in `answers.txt`.

🎮 **Challenge:** Create a "cat and mouse" — implement a protection that detects the `LD_PRELOAD` bypass, then find a way to defeat that too.

LAB 12: Process Management

🎯 Mission: "The Process Orchestrator"

> ***Story:*** *You're building a service manager that can launch, monitor, and control child processes. You need to master* `fork()` , `exec()` , `wait()` *, signals, and process groups.*

📖 Background

Process Lifecycle:

```
   fork()          exec()          exit()
Parent ──────> Child ──────> New Program ──────> Zombie
   │             │                                 │
   │             │                                 │
   └─────────────┴─── waitpid() ──────────────────┘
                      (reaps zombie)
```

Key System Calls:

Call	Purpose
`fork()`	Create child process (copy of parent)
`exec*()`	Replace process image with new program
`wait()/waitpid()`	Wait for child state change
`getpid()/getppid()`	Get process/parent PID
`setpgid()`	Set process group
`setsid()`	Create new session
`kill()`	Send signal to process
`signal()/sigaction()`	Register signal handlers

🏁 Exercise 12.1: Fork, Exec, and Wait

Objective: Master process creation patterns.

Step 1: Examine and build `process_lab.c` :

```
cd code/system_programming/lab07_process_management/
gcc -Wall -g process_lab.c -o process_lab
./process_lab
```

Step 2: Observe the fork() behavior:

```
# The program demonstrates:
# - fork() creating child processes
# - exec() replacing child with a new program
# - waitpid() reaping children
```

```
# - Orphan and zombie processes

# Monitor processes in another terminal:
watch -n 0.5 "ps aux | grep process_lab"
```

Step 3: Experiment with process states:

```
# Create a zombie intentionally
gcc -g zombie_creator.c -o zombie_creator
./zombie_creator &
ps aux | grep defunct     # See the zombie!
# Then observe what happens after parent exits
```

🏁 Exercise 12.2: Signal Handling

Objective: Implement robust signal handling for a daemon-like process.

Step 1: Examine and build `signal_handler.c`:

```
gcc -Wall -g signal_handler.c -o signal_handler
./signal_handler &
SH_PID=$!
```

Step 2: Send various signals:

```
kill -USR1 $SH_PID     # Custom action
kill -USR2 $SH_PID     # Custom action
kill -HUP $SH_PID      # Reload configuration
kill -TERM $SH_PID     # Graceful shutdown
```

Step 3: Observe signal mask and pending signals:

```
cat /proc/$SH_PID/status | grep -i sig
```

🎮 **Challenge:** Build a `mini_supervisor` program that:

1. Launches a child process
2. Monitors it with `waitpid()`
3. Automatically restarts it if it crashes
4. Implements a backoff strategy (wait 1s, 2s, 4s, 8s between restarts)
5. Handles `SIGTERM` for graceful shutdown of both parent and child

---

🏁 Exercise 12.3: Process Management Commands

Objective: Master the command-line tools for process management.

Complete these exercises and record your commands:

```
# 1. List all processes in tree format
pstree -p | head -30

# 2. Find the top 5 CPU-consuming processes
ps aux --sort=-%cpu | head -6
```

```
# 3. Find all processes owned by your user
ps -u $(whoami) -o pid,ppid,stat,%cpu,%mem,cmd

# 4. Run a command with modified scheduling priority
nice -n 10 sleep 100 &
renice -n 5 -p $!

# 5. Run a command in the background, immune to hangup
nohup ./long_running_task &

# 6. Move a foreground process to background
# (Start a process, Ctrl+Z, then bg)

# 7. Examine process limits
cat /proc/$$/limits

# 8. Set resource limits
ulimit -v 1048576     # Limit virtual memory to 1GB
ulimit -u 100         # Limit user processes to 100
```

LAB 13: File Operations

🎯 Mission: "The File Systems Specialist"

> ***Story:*** *You're implementing a log rotation system. It needs to create, read, write, copy, monitor, and manage files efficiently using low-level system calls.*

📖 Background

File I/O Layers:

```
Application
    │
    ├── fopen/fread/fwrite (stdio – buffered, C library)
    │
    ├── open/read/write    (POSIX – unbuffered, syscalls)
    │
    ├── mmap               (Memory-mapped I/O)
    │
    └── io_uring / aio     (Async I/O)

Each layer has different performance characteristics.
```

🏁 Exercise 13.1: Low-Level File I/O

Objective: Use POSIX file operations (open/read/write/lseek/close) directly.

Step 1: Examine and build `file_ops.c`:

```
cd code/system_programming/lab06_file_operations/
gcc -Wall -g file_ops.c -o file_ops
./file_ops
```

Step 2: The program demonstrates:

- `open()` with various flags (`O_RDONLY` , `O_WRONLY` , `O_CREAT` , `O_APPEND` , `O_TRUNC`)
- `read()` / `write()` with error handling
- `lseek()` for random access
- `fstat()` for file metadata
- `ftruncate()` for resizing
- `fcntl()` for file locking

Step 3: Compare buffered vs unbuffered I/O performance:

```
gcc -Wall -g file_benchmark.c -o file_bench
./file_bench
# Writes 100MB using: fwrite, write(big), write(small), mmap
```

🏁 Exercise 13.2: Memory-Mapped I/O and File Monitoring

Objective: Use `mmap()` for efficient file access and `inotify` for monitoring.

Step 1: Examine and build `mmap_demo.c` :

```
gcc -Wall -g mmap_demo.c -o mmap_demo
./mmap_demo
```

Step 2: Build the file watcher:

```
gcc -Wall -g file_watcher.c -o file_watcher
./file_watcher /tmp/watched_dir &

# In another terminal:
mkdir -p /tmp/watched_dir
touch /tmp/watched_dir/test.txt
echo "hello" > /tmp/watched_dir/test.txt
rm /tmp/watched_dir/test.txt
# Observe the watcher output
```

🎮 **Challenge:** Build a `tail -f` clone using `inotify` + `read()` :

- Watch a file for modifications
- When modified, read and print only new content
- Handle file truncation and rotation

LAB 14: Profiling

🎯 Mission: "The Performance Detective"

> ***Story:*** *A critical application is 10x slower than expected. You must use profiling tools to identify the bottleneck, then prove the fix works.*

📖 Background

Profiling Tools:

Tool	Type	What It Measures
`gprof`	Instrumentation	Function call counts, time per function

`perf`	Sampling	CPU cycles, cache misses, branch mispredicts
`valgrind --tool=callgrind`	Instrumentation	Instruction-level profiling
`time`	Wall clock	Total real/user/sys time
`strace -c`	Syscall	System call counts and time
`ltrace -c`	Library	Library call counts and time
`/usr/bin/time -v`	Resource	Memory, page faults, context switches

🏁 Exercise 14.1: Profiling with gprof

Objective: Find the bottleneck function using gprof.

Step 1: Build the profiling target with instrumentation:

```
cd code/system_programming/lab16_profiling/
gcc -pg -g profile_target.c -o profile_target -lm
```

Step 2: Run to generate profile data:

```
./profile_target
ls -la gmon.out     # Profile data file
```

Step 3: Analyze the profile:

```
gprof profile_target gmon.out > profile_report.txt
cat profile_report.txt
```

Step 4: Identify the bottleneck:

```
Flat profile — top time consumers:
  %time   cumulative   self     calls   function
  ___     ___          ___      ___     ___________
```

Step 5: Fix the bottleneck, re-profile, and verify the improvement.

🏁 Exercise 14.2: Profiling with perf

Objective: Use perf for hardware-level performance analysis.

```
# Record performance data
sudo perf record -g ./profile_target

# Display report
sudo perf report

# Quick statistics
sudo perf stat ./profile_target

# Top functions by CPU usage
sudo perf top -p $(pgrep profile_target)
```

🎮 **Challenge:** Use `perf stat` to compare the cache miss rates of a program that accesses an array sequentially vs randomly:

```
sudo perf stat -e cache-misses,cache-references ./sequential_access
sudo perf stat -e cache-misses,cache-references ./random_access
```

LAB 15: Thread Management

🎯 Mission: "The Concurrency Engineer"

> ***Story:*** *Your team is parallelizing a computationally expensive image processing pipeline. You need to manage thread creation, synchronization, and safe data sharing using POSIX threads.*

📖 Background

POSIX Threads (pthreads) API:

Function	Purpose
`pthread_create()`	Create a new thread
`pthread_join()`	Wait for thread to finish
`pthread_detach()`	Detach thread (auto-cleanup)
`pthread_exit()`	Terminate calling thread
`pthread_mutex_lock/unlock()`	Mutual exclusion
`pthread_rwlock_*()`	Reader-writer lock
`pthread_cond_wait/signal()`	Condition variable
`pthread_barrier_wait()`	Barrier synchronization
`pthread_key_create()`	Thread-local storage

Thread Synchronization Primitives:

```
MUTEX                RWLOCK               CONDITION VAR
┌─────────┐          ┌─────────┐          ┌─────────┐
│ 1 thread│          │ N read  │          │ wait()  │
│ at a    │          │ OR      │          │ signal()│
│ time    │          │ 1 write │          │ bcast() │
└─────────┘          └─────────┘          └─────────┘

BARRIER              SEMAPHORE
┌─────────┐          ┌─────────┐
│ All N   │          │ Count   │
│ arrive  │          │ up/down │
│ to pass │          │ 0=block │
└─────────┘          └─────────┘
```

🏁 Exercise 15.1: Thread Basics and Synchronization

Objective: Create, synchronize, and manage threads.

Step 1: Examine and build `thread_lab.c` :

```
cd code/system_programming/lab08_thread_management/
gcc -Wall -g -pthread thread_lab.c -o thread_lab
./thread_lab
```

Step 2: The program demonstrates:

- Creating multiple threads
- Passing data to threads
- Race condition (without mutex)
- Fixed version (with mutex)
- Thread-local storage

Step 3: Observe the race condition:

```
# Run multiple times — inconsistent results without mutex
for i in $(seq 1 10); do ./thread_lab race; done

# Now with mutex — consistent
for i in $(seq 1 10); do ./thread_lab safe; done
```

🏁 Exercise 15.2: Producer-Consumer with Condition Variables

Objective: Implement the classic producer-consumer pattern.

Step 1: Examine and build `producer_consumer.c`:

```
gcc -Wall -g -pthread producer_consumer.c -o prodcon
./prodcon
```

Step 2: The program implements:

- A bounded buffer (ring buffer)
- Multiple producer threads
- Multiple consumer threads
- Condition variables for full/empty signaling
- Graceful shutdown

Step 3: Test with different configurations:

```
./prodcon 4 2 1000    # 4 producers, 2 consumers, 1000 items each
./prodcon 1 4 5000    # 1 producer, 4 consumers, 5000 items
```

🏁 Exercise 15.3: Thread Pool Implementation

Objective: Build a reusable thread pool for task execution.

Step 1: Examine and build `thread_pool.c`:

```
gcc -Wall -g -pthread thread_pool.c -o thread_pool_demo
./thread_pool_demo
```

Step 2: The thread pool supports:

- Configurable number of worker threads
- Task queue with mutex protection

- Condition variable for task notification
- Graceful shutdown with `pool_destroy()`

🎮 **Challenge:** Extend the thread pool to support:

1. Task priorities (high/medium/low)
2. A `pool_wait()` function that blocks until all submitted tasks complete
3. Dynamic resizing — add/remove worker threads at runtime

PART 3 — Advanced System Programming

LAB 16: Inter-Process Communication (IPC)

🎯 Mission: "The Message Network"

> ***Story:*** *Your team is building a distributed control system where multiple processes must coordinate in real-time. You must master every IPC mechanism Linux offers — from classic pipes to modern shared memory — to design a robust communication fabric.*

📖 Background

Linux provides several IPC mechanisms, each with different characteristics:

Mechanism	Direction	Persistence	Speed	Best For
Pipe	Unidirectional	Process lifetime	Medium	Parent-child
Named Pipe (FIFO)	Unidirectional	Filesystem	Medium	Unrelated processes
Message Queue	Bidirectional	Kernel	Medium	Structured messages
Shared Memory	Bidirectional	Kernel/filesystem	Fastest	Large data sharing
Semaphore	Synchronization	Kernel	Fast	Access coordination
Unix Domain Socket	Bidirectional	Filesystem	Fast	Complex protocols
eventfd	Notification	Process lifetime	Very fast	Event signaling

🏁 Exercise 16.1: Pipes and Named Pipes

Objective: Use anonymous pipes for parent-child communication and FIFOs for unrelated processes.

Step 1: Examine and build the pipe demo:

```
cd code/system_programming/lab09_ipc/
cat pipe_demo.c
gcc -Wall -g pipe_demo.c -o pipe_demo
./pipe_demo
```

Step 2: The program demonstrates:

- Creating anonymous pipes with `pipe()`
- Bidirectional communication using two pipes
- `dup2()` to redirect stdin/stdout through pipes
- Non-blocking pipe reads with `fcntl()`

Step 3: Test the named pipe (FIFO) program:

```
gcc -Wall -g fifo_demo.c -o fifo_demo
# Terminal 1:
./fifo_demo server
# Terminal 2:
./fifo_demo client
```

Step 4: Observe that FIFOs persist in the filesystem:

```
ls -la /tmp/my_fifo
file /tmp/my_fifo
```

🏁 Exercise 16.2: POSIX Message Queues

Objective: Use message queues for structured, prioritized communication.

Step 1: Build and run the message queue demo:

```
gcc -Wall -g mqueue_demo.c -o mqueue_demo -lrt
./mqueue_demo
```

Step 2: The program demonstrates:

- `mq_open()`, `mq_send()`, `mq_receive()`, `mq_close()`, `mq_unlink()`
- Message priorities
- Non-blocking receives
- `mq_notify()` for asynchronous notification

Step 3: Inspect the message queue:

```
# List all POSIX message queues
ls /dev/mqueue/
cat /dev/mqueue/my_queue   # Shows queue attributes
```

🏁 Exercise 16.3: Shared Memory and Semaphores

Objective: Share data between processes at memory speed using POSIX shared memory, synchronized with semaphores.

Step 1: Build and run:

```
gcc -Wall -g shm_demo.c -o shm_demo -lrt -lpthread
./shm_demo
```

Step 2: The program demonstrates:

- `shm_open()`, `ftruncate()`, `mmap()` for shared memory
- `sem_open()`, `sem_wait()`, `sem_post()` for POSIX semaphores
- Multiple producers/consumers sharing a ring buffer via shared memory
- Proper cleanup with `shm_unlink()` and `sem_unlink()`

Step 3: Inspect shared memory objects:

```
ls /dev/shm/
```

🏁 Exercise 16.4: Unix Domain Sockets

Objective: Build a client-server system using Unix domain sockets.

Step 1: Build and run:

```
gcc -Wall -g unix_socket_demo.c -o unix_socket_demo -lpthread
./unix_socket_demo
```

Step 2: The program demonstrates:

- `socket(AF_UNIX, SOCK_STREAM, 0)` for stream sockets
- `bind()`, `listen()`, `accept()` on server side
- `connect()`, `send()`, `recv()` on client side
- Passing file descriptors between processes via `sendmsg()` / `recvmsg()` with `SCM_RIGHTS`

💡 Challenge

Build an IPC benchmark that compares the throughput of pipes, message queues, shared memory, and Unix sockets for transferring 100MB of data. Create a summary table of results.

LAB 17: Process Scheduling

🎯 Mission: "The Priority Controller"

> ***Story:*** *Your real-time control system has tasks of varying importance. Some must never be delayed. You must learn to control the Linux process scheduler to ensure critical tasks get the CPU time they need.*

📖 Background

Linux supports multiple scheduling policies:

Policy	Type	Priority Range	Description
`SCHED_OTHER` (CFS)	Normal	0 (nice: -20 to 19)	Default — fair sharing
`SCHED_FIFO`	Real-time	1–99	First-in, first-out (preempts lower)
`SCHED_RR`	Real-time	1–99	Round-robin with time quantum
`SCHED_BATCH`	Normal	0	CPU-intensive batch jobs
`SCHED_IDLE`	Normal	0	Very low priority
`SCHED_DEADLINE`	Real-time	—	Earliest-deadline-first

🏁 Exercise 17.1: Scheduling Policies and Priorities

Objective: Observe and modify process scheduling behavior.

Step 1: Build and run the scheduling demo:

```
cd code/system_programming/lab10_scheduling/
gcc -Wall -g sched_demo.c -o sched_demo -lpthread
sudo ./sched_demo
```

Step 2: The program demonstrates:

- `sched_getscheduler()` / `sched_setscheduler()` to query/set policies
- `sched_getparam()` / `sched_setparam()` for priorities
- `nice()` and `setpriority()` for CFS niceness
- `sched_get_priority_min()` / `sched_get_priority_max()` for valid ranges
- Comparison of SCHED_FIFO vs SCHED_RR vs CFS under CPU contention

Step 3: Observe scheduling from the command line:

```
# Check current process scheduling
chrt -p $$
# Run a command with real-time priority
sudo chrt -f 50 ./my_program
# Run with specific nice value
nice -n -10 ./my_program
# Change nice of a running process
renice -n 5 -p <PID>
```

Step 4: Compare behavior:

```
# Spawn CPU-bound tasks at different nice levels and observe
nice -n 19 ./cpu_burner &    # Low priority
nice -n -20 ./cpu_burner &   # High priority (needs sudo)
top    # Observe CPU% differences
```

💡 Challenge

Write a program that creates 5 child processes, each with a different scheduling policy/priority, and demonstrates that higher-priority real-time tasks preempt lower-priority ones.

LAB 18: Thread Scheduling

🎯 Mission: "The Thread Orchestrator"

> ***Story:*** *Your control system runs multiple threads with different deadlines. Some threads handle sensor data (real-time), others do logging (background). You must configure each thread's scheduling independently.*

📖 Background

Threads can have individual scheduling policies independent of the process:

```
// Set scheduling via thread attributes (at creation)
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
pthread_attr_setschedparam(&attr, &param);
pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);

// Or change at runtime
pthread_setschedparam(thread, SCHED_FIFO, &param);
```

Key concepts:

- **Contention scope:** `PTHREAD_SCOPE_SYSTEM` (kernel-scheduled, 1:1) vs `PTHREAD_SCOPE_PROCESS` (user-level)
- **Inherit vs explicit:** Threads can inherit parent's scheduling or set their own
- **Priority inversion:** When a high-priority thread waits for a lock held by a low-priority thread
- **Priority inheritance mutex:** Prevents priority inversion

🏁 Exercise 18.1: Thread Scheduling Policies

Objective: Create threads with different scheduling policies and observe behavior.

Step 1: Build and run:

```
cd code/system_programming/lab11_thread_sched/
gcc -Wall -g thread_sched.c -o thread_sched -lpthread
sudo ./thread_sched
```

Step 2: The program demonstrates:

- Setting per-thread scheduling policy with `pthread_attr_setschedpolicy()`
- Using `PTHREAD_EXPLICIT_SCHED` to override inherited scheduling
- Real-time threads preempting normal threads
- Priority inversion and `PTHREAD_PRIO_INHERIT` mutex protocol
- `pthread_setschedparam()` to change priority at runtime

Step 3: Monitor thread scheduling:

```
# View thread info for a process
ps -eLo pid,lwp,nlwp,cls,pri,nice,comm | head -30
# 'cls' shows scheduling class: TS=SCHED_OTHER, FF=SCHED_FIFO, RR=SCHED_RR
```

💡 Challenge

Build a simulation with 3 threads that demonstrates priority inversion: a high-priority thread blocked by a low-priority lock holder while a medium-priority thread runs. Then fix it with a priority inheritance mutex.

LAB 19: Users and Groups

🎯 Mission: "The Identity Manager"

> ***Story:*** *Your system manages access control for a multi-user embedded device. You must understand how Linux identifies users and groups, and how to query and manage these identities programmatically.*

📖 Background

Linux identity system:

- **UID** (User ID) — unique numeric ID for each user
- **GID** (Group ID) — primary group for each user
- **Supplementary groups** — additional group memberships
- **Effective vs Real IDs** — the effective ID determines permissions
- **SUID/SGID** — programs that run with the file owner's privileges

Key files: `/etc/passwd`, `/etc/group`, `/etc/shadow`

🏁 Exercise 19.1: User and Group Information

Objective: Query and manipulate user/group identity programmatically.

Step 1: Build and run:

```
cd code/system_programming/lab12_users_groups/
gcc -Wall -g user_info.c -o user_info
```

```
./user_info
```

Step 2: The program demonstrates:

- `getuid()`, `geteuid()`, `getgid()`, `getegid()` for current IDs
- `getpwuid()`, `getpwnam()` for `/etc/passwd` lookups
- `getgrgid()`, `getgrnam()` for `/etc/group` lookups
- `getgroups()` for supplementary group list
- `setuid()`, `setgid()`, `seteuid()`, `setegid()` for privilege changes
- Capability checking with `cap_get_proc()` (libcap)

Step 3: Explore from the command line:

```
id                          # Current user's UID, GID, groups
id -u                       # Just UID
id -G                       # All group IDs
groups                      # Group names
cat /etc/passwd | head      # User database
cat /etc/group | head       # Group database
getent passwd username      # NSS-aware lookup
```

🏁 Exercise 19.2: Privilege Dropping

Objective: Write a program that starts as root and drops privileges safely.

Step 1: Build and test:

```
gcc -Wall -g priv_drop.c -o priv_drop
sudo ./priv_drop
```

Step 2: The program demonstrates:

- Starting as root (UID 0)
- Setting supplementary groups with `setgroups()`
- Dropping GID with `setgid()` / `setegid()`
- Dropping UID with `setuid()` / `seteuid()`
- Verifying privileges cannot be re-escalated
- Why the order (groups → GID → UID) matters

💡 Challenge

Write a SUID-aware program that checks if it's running with elevated privileges, performs a privileged operation (e.g., reading `/etc/shadow`), then drops privileges permanently before processing data.

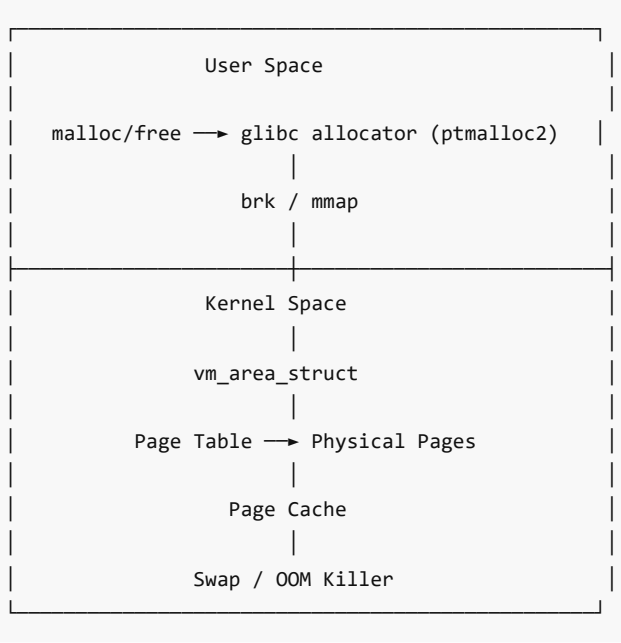
LAB 20: Memory Management

🎯 Mission: "The Memory Architect"

> ***Story:*** *Your embedded system has limited RAM. You must master Linux memory management — from `malloc()` internals to the kernel's virtual memory system — to write memory-efficient code and diagnose leaks.*

📖 Background

Linux memory management hierarchy:



Key concepts:

- **Virtual address space:** `/proc/<pid>/maps` shows all mapped regions
- **brk()** extends the data segment (small allocations)
- **mmap()** creates new memory regions (large allocations, >128KB by default)
- **Overcommit:** Linux allows allocating more virtual memory than physical RAM
- **Page fault:** Accessing a page not yet backed by physical memory

🏁 Exercise 20.1: Memory Allocation Internals

Objective: Understand how `malloc()`, `brk()`, and `mmap()` work under the hood.

Step 1: Build and run:

```
cd code/system_programming/lab13_memory/
gcc -Wall -g memory_demo.c -o memory_demo
./memory_demo
```

Step 2: The program demonstrates:

- `brk()` / `sbrk()` to manually extend the heap
- `malloc()` behavior for small vs large allocations
- Watching `/proc/self/maps` change as memory is allocated
- `mmap(MAP_ANONYMOUS)` for direct kernel page allocation
- `madvise()` hints (MADV_SEQUENTIAL, MADV_WILLNEED, MADV_DONTNEED)
- `mlock()` / `mlockall()` to pin pages in RAM

Step 3: Monitor memory from the command line:

```
# Process memory map
cat /proc/<pid>/maps
# Memory stats
cat /proc/<pid>/status | grep -i vm
# System memory overview
cat /proc/meminfo
free -h
```

🏁 Exercise 20.2: Memory Leak Detection

Objective: Use Valgrind and custom tracking to detect memory leaks.

Step 1: Build a deliberately leaky program:

```
gcc -Wall -g memory_leaks.c -o memory_leaks
```

Step 2: Run with Valgrind:

```
valgrind --leak-check=full --show-leak-kinds=all ./memory_leaks
valgrind --tool=massif ./memory_leaks   # Heap profiler
ms_print massif.out.<pid>               # Visualize heap usage over time
```

Step 3: The program contains several leak patterns:

- Direct memory leak (malloc without free)
- Indirect leak (lost pointer to allocated block)
- Still-reachable memory (not freed at exit)
- Double-free scenario
- Use-after-free scenario

💡 Challenge

Write a custom memory allocator wrapper that tracks all allocations/frees and reports leaks at program exit with file/line information using `__FILE__` and `__LINE__` macros.

LAB 21: OOM Killer

🎯 Mission: "Surviving the OOM Storm"

> ***Story:*** *Production servers are mysteriously killing processes. You must understand how the Linux Out-Of-Memory (OOM) killer decides which process to terminate, and learn to protect critical processes from being killed.*

📖 Background

When the system runs out of memory and swap:

1. The kernel's OOM killer activates
2. It scores each process (higher = more likely to be killed)
3. The process with the highest score is killed with `SIGKILL`

OOM score factors:

- RSS (Resident Set Size) — processes using more RAM score higher
- `oom_score_adj` — user-adjustable bias (−1000 to +1000)
- `oom_score_adj = -1000` makes a process OOM-immune
- Root processes are slightly favored

🏁 Exercise 21.1: Understanding and Controlling the OOM Killer

Objective: Observe OOM scores and adjust them.

Step 1: Build and run:

```
cd code/system_programming/lab14_oom/
gcc -Wall -g oom_demo.c -o oom_demo
```

```
./oom_demo
```

Step 2: The program demonstrates:

- Reading `/proc/self/oom_score` and `/proc/self/oom_score_adj`
- Adjusting OOM score with `/proc/<pid>/oom_score_adj`
- Monitoring OOM events via `/dev/kmsg` or `dmesg`
- Allocating memory until OOM triggers (use with caution in a VM!)

Step 3: Explore OOM from the command line:

```
# Check OOM score for all processes (sorted)
for pid in /proc/[0-9]*; do
    name=$(cat $pid/comm 2>/dev/null)
    score=$(cat $pid/oom_score 2>/dev/null)
    adj=$(cat $pid/oom_score_adj 2>/dev/null)
    echo "$score $adj $name $(basename $pid)"
done | sort -rn | head -20

# Make a process OOM-immune (requires root)
echo -1000 | sudo tee /proc/<pid>/oom_score_adj

# Check recent OOM kills
dmesg | grep -i "oom\|killed process"
```

Step 4: Understand the overcommit settings:

```
cat /proc/sys/vm/overcommit_memory    # 0=heuristic, 1=always, 2=never
cat /proc/sys/vm/overcommit_ratio     # percentage for mode 2
```

💡 Challenge

Write a program that spawns multiple child processes, each consuming increasing amounts of memory. Monitor which child gets OOM-killed first. Then set `oom_score_adj` to protect the most important child and observe the change.

LAB 22: Temporary Files

🎯 Mission: "The Secure Scratch Pad"

> ***Story:*** *Your application needs temporary storage for intermediate computations. But temp files are a common attack vector — race conditions, symlink attacks, and data leaks lurk in* `/tmp` *. You must learn to create temp files securely.*

📖 Background

Insecure temp file patterns and their secure alternatives:

Insecure ❌	Vulnerability	Secure ✅
`tmpnam()`	Race condition (TOCTOU)	`mkstemp()`
`tempnam()`	Predictable names	`mkostemp()`
`fopen("/tmp/myapp.tmp")`	Symlink attack	`tmpfile()`
Fixed name in `/tmp`	Overwrite / read by others	`mkdtemp()` for directories

Key principles:

- Always use `mkstemp()` or `tmpfile()` — they create and open atomically
- Set restrictive permissions (0600)
- Use `O_EXCL | O_CREAT` to prevent symlink following
- Unlink temp files as soon as possible
- Consider `O_TMPFILE` flag (Linux 3.11+) for truly anonymous temp files

🏁 Exercise 22.1: Secure Temporary File Handling

Objective: Compare insecure and secure temp file creation methods.

Step 1: Build and run:

```
cd code/system_programming/lab15_tempfiles/
gcc -Wall -g tempfile_demo.c -o tempfile_demo
./tempfile_demo
```

Step 2: The program demonstrates:

- `mkstemp()` — secure temp file creation
- `mkostemp()` — with additional flags (O_APPEND, O_CLOEXEC)
- `mkdtemp()` — secure temp directory creation
- `tmpfile()` — anonymous temp file (auto-deleted on close)
- `O_TMPFILE` — Linux anonymous file (never has a name)
- Demonstrating the TOCTOU race condition with the insecure approach

Step 3: Inspect temp file security:

```
ls -la /tmp/
# Note file permissions — should be 0600 for secure temp files
stat /tmp/myapp.*
```

💡 Challenge

Write a program that demonstrates a symlink attack on an insecure temp file, then show how `mkstemp()` prevents it.

LAB 23: Timers

🎯 Mission: "The Precision Clock"

> ***Story:*** *Your real-time system needs precise timing — periodic tasks, one-shot timeouts, and watchdog timers. You must master Linux timer APIs from basic* `sleep()` *to high-resolution POSIX timers.*

📖 Background

Linux timer APIs (from simple to advanced):

API	Resolution	Type	Best For
`sleep()`	1 second	Blocking	Coarse delays
`usleep()`	1 microsecond	Blocking	Short delays
`nanosleep()`	1 nanosecond	Blocking	Precise delays

`alarm()`	1 second	Signal-based	Simple timeouts
`setitimer()`	1 microsecond	Signal-based	Periodic tasks
`timer_create()`	1 nanosecond	Signal/thread	Precise periodic
`timerfd_create()`	1 nanosecond	File descriptor	Poll/epoll integration

🏁 Exercise 23.1: Timer APIs

Objective: Use various Linux timer mechanisms.

Step 1: Build and run:

```
cd code/system_programming/lab16_timers/
gcc -Wall -g timer_demo.c -o timer_demo -lrt -lpthread
./timer_demo
```

Step 2: The program demonstrates:

- `clock_gettime()` with different clocks (CLOCK_REALTIME, CLOCK_MONOTONIC, CLOCK_PROCESS_CPUTIME_ID)
- `nanosleep()` with remainder handling for interrupted sleep
- `timer_create()` / `timer_settime()` for POSIX timers
- `timerfd_create()` / `timerfd_settime()` for file-descriptor-based timers
- One-shot vs periodic timers
- Timer overrun detection with `timer_getoverrun()`

Step 3: Monitor from the command line:

```
# View process timers
cat /proc/<pid>/timers
# High-resolution timer availability
cat /proc/timer_list | head -50
```

💡 Challenge

Build a watchdog timer that monitors a child process. If the child doesn't send a "heartbeat" (SIGUSR1) within 5 seconds, the watchdog kills and restarts it.

LAB 24: FUSE — Filesystem in Userspace

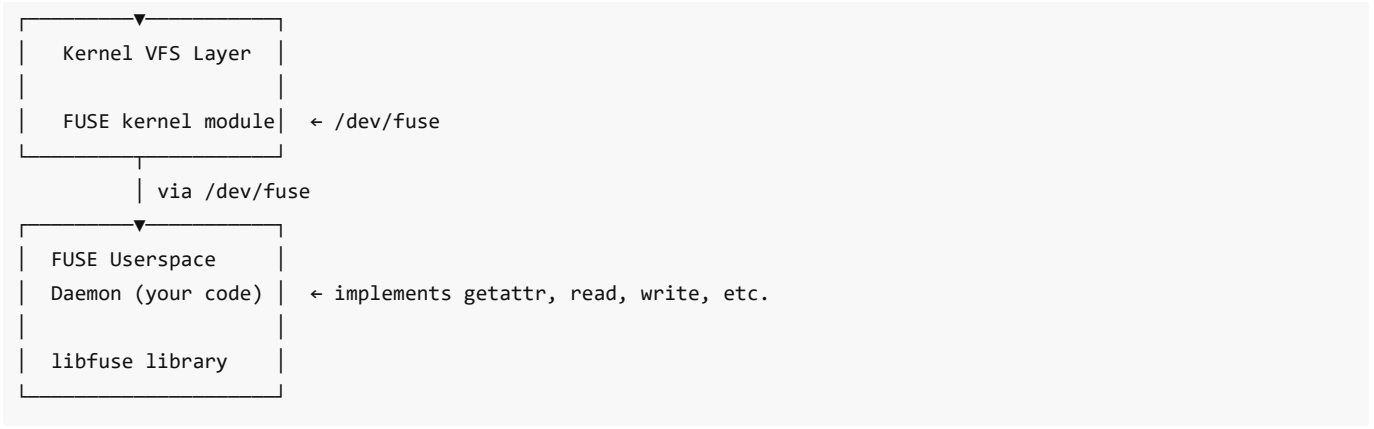
🎯 Mission: "The Custom Filesystem"

> ***Story:*** *You need a custom filesystem that transforms data on-the-fly — but writing a kernel module is risky and complex. FUSE lets you build filesystems as normal userspace programs with full debugging support.*

📖 Background

FUSE architecture:

```
┌─────────────────────┐
│   User Application  │  (ls, cat, cp, etc.)
│   read("/mnt/a.txt")│
└──────────┬──────────┘
           │ VFS syscall
```



Key FUSE operations:

- `getattr()` — equivalent of `stat()`, most frequently called
- `readdir()` — list directory entries
- `open()` / `read()` / `write()` — file I/O
- `mkdir()` / `rmdir()` / `unlink()` — file management
- `create()` — create new files

🏁 Exercise 24.1: Simple In-Memory Filesystem

Objective: Build a basic FUSE filesystem that stores files in memory.

Step 1: Install FUSE development libraries:

```
sudo apt install -y libfuse3-dev fuse3
```

Step 2: Build and mount:

```
cd code/system_programming/lab17_fuse/
gcc -Wall -g memfs.c -o memfs $(pkg-config --cflags --libs fuse3)
mkdir -p /tmp/memfs_mount
./memfs /tmp/memfs_mount
```

Step 3: Test the filesystem:

```
# In another terminal:
ls /tmp/memfs_mount/
echo "Hello FUSE" > /tmp/memfs_mount/test.txt
cat /tmp/memfs_mount/test.txt
mkdir /tmp/memfs_mount/subdir
cp /etc/hostname /tmp/memfs_mount/
ls -la /tmp/memfs_mount/
df -h /tmp/memfs_mount/
```

Step 4: Unmount:

```
fusermount3 -u /tmp/memfs_mount
```

🏁 Exercise 24.2: Filesystem Type Investigation

Objective: Understand different Linux filesystem types and their characteristics.

Step 1: Explore filesystem types on your system:

```
# List mounted filesystems
mount | column -t
df -Th    # Show filesystem types

# Check supported filesystem types
cat /proc/filesystems

# Examine superblock information
sudo dumpe2fs /dev/sda1 2>/dev/null | head -40    # ext4
sudo xfs_info /dev/sda1 2>/dev/null               # xfs

# View filesystem-specific stats
stat -f /    # Filesystem stats for root
```

Step 2: Compare filesystem features:

Feature	ext4	XFS	Btrfs	tmpfs	procfs	sysfs
On-disk	Yes	Yes	Yes	No (RAM)	No (virtual)	No (virtual)
Journaling	Yes	Yes	CoW	N/A	N/A	N/A
Max file size	16 TB	8 EB	16 EB	RAM	N/A	N/A
Snapshots	No	No	Yes	No	No	No
Compression	No	No	Yes	No	No	No

💡 Challenge

Add write support, file deletion, and directory creation to the FUSE filesystem. Implement a "versioned" filesystem that keeps the last 3 versions of each file.

LAB 25: FUSE FAT Filesystem

🎯 Mission: "The FAT Decoder"

> ***Story:*** *An old USB drive has critical data stored on a FAT16 filesystem, but the kernel driver can't mount it due to corruption. You must write a FUSE-based FAT reader that can parse the raw disk image and recover the files.*

📖 Background

FAT filesystem structure on disk:

```
┌─────────────┬─────────────┬─────────────┬─────────────────┐
│ Boot Sector │    FAT 1    │    FAT 2    │    Data Region  │
│ (512 bytes) │ (File Alloc │  (Backup)   │  (Clusters)     │
│             │   Table)    │             │                 │
│ - BPB       │ - Chain of  │             │ - Root Directory │
│ - Geometry  │   clusters  │             │ - File data     │
│ - FAT info  │             │             │                 │
└─────────────┴─────────────┴─────────────┴─────────────────┘
```

FAT directory entry (32 bytes):

```
Offset  Size  Field
0x00    8     Filename (padded with spaces)
0x08    3     Extension
0x0B    1     Attributes (R/H/S/V/D/A)
0x1A    2     First cluster number
0x1C    4     File size
```

🏁 Exercise 25.1: FAT Image Parser

Objective: Parse and display the structure of a FAT filesystem image.

Step 1: Create a test FAT image:

```
cd code/system_programming/lab18_fuse_fat/
# Create a 4MB FAT16 image
dd if=/dev/zero of=test.fat16 bs=1M count=4
mkfs.fat -F 16 -n "TESTDISK" test.fat16

# Mount and add files
mkdir -p /tmp/fat_mount
sudo mount -o loop test.fat16 /tmp/fat_mount
sudo cp /etc/hostname /tmp/fat_mount/
echo "Hello FAT" | sudo tee /tmp/fat_mount/HELLO.TXT
sudo mkdir /tmp/fat_mount/DOCS
echo "Readme content" | sudo tee /tmp/fat_mount/DOCS/README.TXT
sudo umount /tmp/fat_mount
```

Step 2: Build and run the FAT parser:

```
gcc -Wall -g fat_parser.c -o fat_parser
./fat_parser test.fat16
```

Step 3: The parser displays:

- Boot sector / BPB fields
- FAT table entries and cluster chains
- Root directory listing with file attributes
- File content extraction

🏁 Exercise 25.2: FUSE FAT Driver

Objective: Mount a FAT image as a FUSE filesystem.

Step 1: Build the FUSE FAT driver:

```
gcc -Wall -g fuse_fat.c -o fuse_fat $(pkg-config --cflags --libs fuse3)
```

Step 2: Mount the FAT image:

```
mkdir -p /tmp/fusefat_mount
./fuse_fat test.fat16 /tmp/fusefat_mount
```

Step 3: Test:

```
ls -la /tmp/fusefat_mount/
cat /tmp/fusefat_mount/HELLO.TXT
ls -la /tmp/fusefat_mount/DOCS/
cat /tmp/fusefat_mount/DOCS/README.TXT
```

Step 4: Unmount:

```
fusermount3 -u /tmp/fusefat_mount
```

💡 Challenge

Extend the FUSE FAT driver with write support — implement `create()`, `write()`, and `unlink()` operations that modify the FAT image file.

LAB 26: CPU Isolation and CPU Pinning

🎯 Mission: "The Dedicated Core"

> ***Story:*** *Your real-time application suffers from jitter — unpredictable delays caused by the kernel scheduling other tasks on the same CPU core. You must isolate cores for dedicated use and pin critical threads to specific CPUs.*

📖 Background

CPU affinity and isolation:

```
┌──────────────┬──────────────┬──────────────┬──────────────┐
│    Core 0    │    Core 1    │    Core 2    │    Core 3    │
│              │              │              │              │
│ General use  │ General use  │  ISOLATED    │  ISOLATED    │
│ (kernel,     │ (kernel,     │  (your app   │  (your app   │
│  services,   │  services)   │   only)      │   only)      │
│  IRQs)       │              │              │              │
└──────────────┴──────────────┴──────────────┴──────────────┘
```

Mechanisms:

- `sched_setaffinity()` — pin process/thread to specific CPUs
- `CPU_SET` **macros** — manipulate CPU bitmasks
- `isolcpus` **boot param** — isolate cores from general scheduler
- `taskset` **command** — set CPU affinity from command line
- **cgroups cpuset** — advanced CPU and memory isolation

🏁 Exercise 26.1: CPU Affinity and Pinning

Objective: Pin processes and threads to specific CPU cores.

Step 1: Build and run:

```
cd code/system_programming/lab19_cpu_pinning/
gcc -Wall -g cpu_pinning.c -o cpu_pinning -lpthread
./cpu_pinning
```

Step 2: The program demonstrates:

- `sched_getaffinity()` / `sched_setaffinity()` for process-level pinning

- `pthread_setaffinity_np()` for thread-level pinning
- `CPU_SET()`, `CPU_ZERO()`, `CPU_ISSET()`, `CPU_COUNT()` macros
- Measuring performance improvement with CPU pinning
- Detecting CPU topology (cores, threads, NUMA nodes)

Step 3: Use `taskset` from the command line:

```
# Run on specific CPUs
taskset -c 0 ./my_program          # CPU 0 only
taskset -c 0,2 ./my_program        # CPUs 0 and 2
taskset -c 0-3 ./my_program        # CPUs 0,1,2,3

# Change affinity of running process
taskset -cp 2 <PID>

# Check current affinity
taskset -cp <PID>
```

🏁 Exercise 26.2: CPU Isolation

Objective: Isolate CPUs and measure the jitter reduction.

Step 1: Build and run the jitter measurement tool:

```
gcc -Wall -g -O2 jitter_measure.c -o jitter_measure -lpthread
# Without isolation:
./jitter_measure 0
# With CPU pinning:
taskset -c 2 ./jitter_measure 2
```

Step 2: The tool measures:

- Loop iteration timing variance (jitter)
- Worst-case latency
- Histogram of timing deviations
- Comparison between isolated and non-isolated CPUs

Step 3: For full isolation (boot parameter — do this in a VM):

```
# Add to kernel command line (e.g., in /etc/default/grub):
# GRUB_CMDLINE_LINUX="isolcpus=2,3 nohz_full=2,3 rcu_nocbs=2,3"
# Then: sudo update-grub && sudo reboot

# Verify isolation
cat /sys/devices/system/cpu/isolated
cat /sys/devices/system/cpu/online
```

💡 Challenge

Write a benchmark that measures the difference in worst-case latency between: (1) normal scheduling, (2) CPU-pinned with `taskset`, (3) CPU-pinned with `SCHED_FIFO` priority 99, and (4) fully isolated CPU with `isolcpus`. Present results in a table.

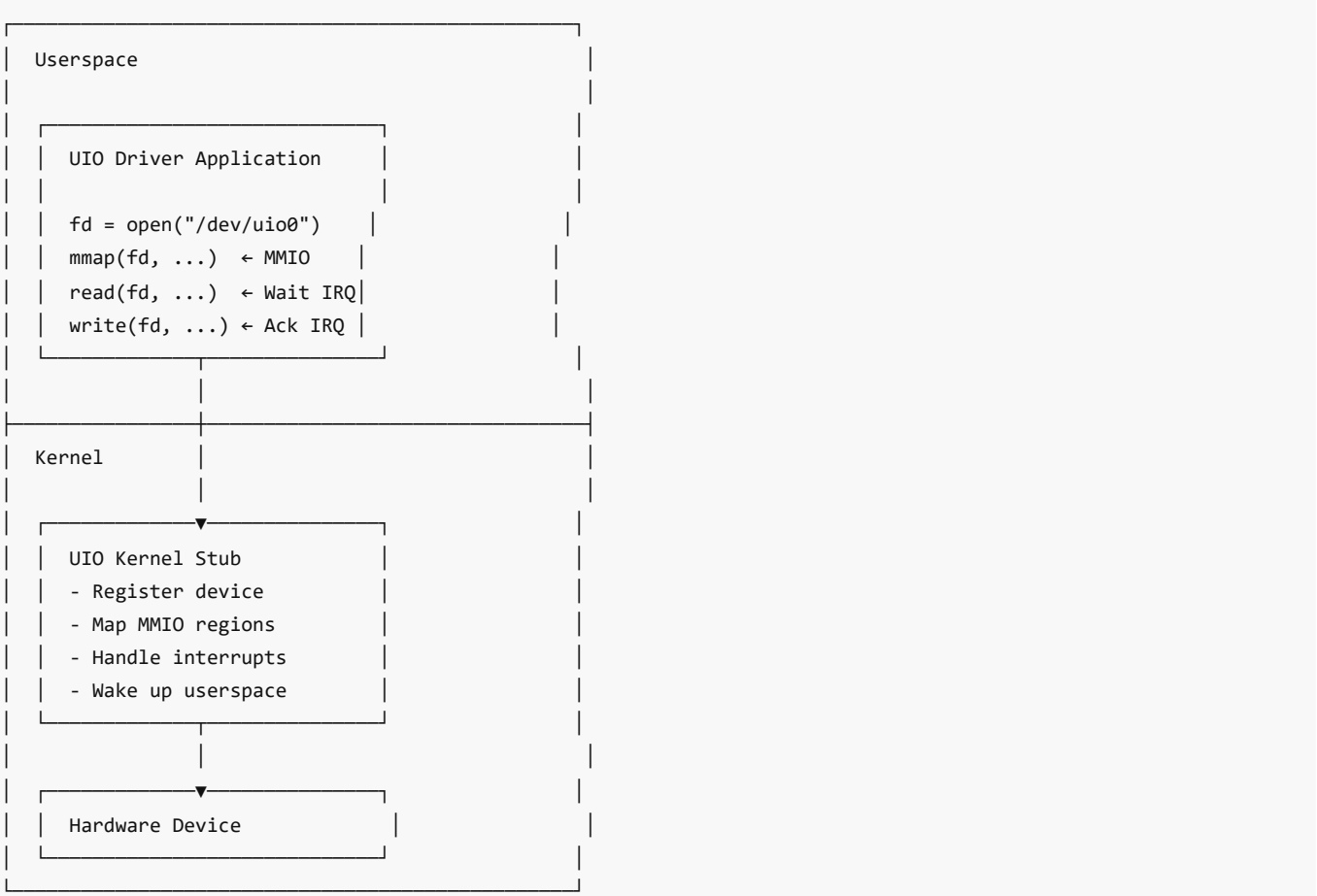
LAB 27: UIO — Userspace I/O Driver

🎯 Mission: "The Userspace Driver"

> ***Story:*** *Your team needs to write a driver for a custom hardware device, but kernel module development is slow and crash-prone. UIO (Userspace I/O) lets you write most of the driver logic in userspace, with only a tiny kernel stub for interrupt handling.*

📖 Background

UIO architecture:



UIO advantages:

- Full C library and debugging tools available
- Crash in userspace doesn't crash the kernel
- No GPL licensing requirements for userspace code
- Faster development cycle (no kernel recompilation)

🏁 Exercise 27.1: UIO Kernel Module

Objective: Create a minimal UIO kernel module and userspace driver.

Step 1: Install kernel headers:

```
sudo apt install -y linux-headers-$(uname -r)
```

Step 2: Build the UIO kernel module:

```
cd code/system_programming/lab20_uio/
make -C kmod/
```

```
sudo insmod kmod/uio_demo.ko
```

Step 3: Verify the UIO device:

```
ls /dev/uio*
cat /sys/class/uio/uio0/name
cat /sys/class/uio/uio0/version
cat /sys/class/uio/uio0/maps/map0/size
```

Step 4: Build and run the userspace driver:

```
gcc -Wall -g uio_user.c -o uio_user
sudo ./uio_user /dev/uio0
```

Step 5: The program demonstrates:

- Opening `/dev/uio0`
- Memory-mapping device registers with `mmap()`
- Reading/writing device registers through the mapped memory
- Waiting for interrupts with `read()` on the UIO file descriptor
- Acknowledging interrupts with `write()`

Step 6: Clean up:

```
sudo rmmod uio_demo
```

🏁 Exercise 27.2: UIO with Simulated Hardware

Objective: Use the `uio_pdrv_genirq` generic UIO platform driver.

Step 1: Build and load the simulated device:

```
cd code/system_programming/lab20_uio/
make -C sim_device/
sudo insmod sim_device/uio_sim.ko
```

Step 2: Run the userspace driver:

```
gcc -Wall -g uio_sim_user.c -o uio_sim_user
sudo ./uio_sim_user
```

Step 3: The program demonstrates:

- Reading simulated sensor data from MMIO registers
- Writing control commands to MMIO registers
- Polling for interrupts from the simulated device
- A complete read/process/respond device driver loop in userspace

💡 Challenge

Write a userspace driver that implements a virtual GPIO controller via UIO. It should expose 8 GPIO pins that can be set as input/output, read/written, and trigger interrupts on state changes.

MINI-PROJECTS

Mini-Project 1: "The Secure Build Pipeline"

Build an end-to-end secure compilation and analysis pipeline that:

1. **Compiles** a C project through all GCC stages with verbose output
2. **Optimizes** using PGO based on test workloads
3. **Encrypts** the final binary with AES-256
4. **Signs** it with an RSA key
5. **Analyzes** and generates a full binary report
6. **Verifies** integrity before deployment

Project Structure

```
project_secure_pipeline/
├── Makefile
├── src/
│   └── app.c              ← Your application
├── keys/
│   ├── private_key.pem    ← Generated RSA private key
│   └── public_key.pem     ← Generated RSA public key
├── build/
│   ├── app.i              ← Preprocessed
│   ├── app.s              ← Assembly
│   ├── app.o              ← Object
│   ├── app                ← Executable
│   ├── app.enc            ← Encrypted binary
│   ├── app.sig            ← Digital signature
│   └── app.sha256         ← SHA-256 checksum
├── profile_data/          ← PGO data
└── reports/
    └── analysis_report.txt ← Binary analysis report
```

Requirements

Phase 1: The Build

- Write a Makefile that compiles through each stage with verbose output
- Include targets: `preprocess`, `compile`, `assemble`, `link`, `all`

Phase 2: Optimization

- Add PGO targets: `pgo-instrument`, `pgo-train`, `pgo-optimize`
- Compare PGO vs non-PGO builds and log results

Phase 3: Security

- Generate RSA-2048 key pair
- Encrypt the binary with AES-256-CBC
- Sign the binary with RSA
- Create SHA-256 checksum

Phase 4: Analysis

- Generate comprehensive binary analysis report
- Include: ELF headers, section info, symbols, strings, dependencies
- Add a `verify` target that checks signature and checksum

Deliverables

1. Complete working Makefile
2. Source code (`app.c`)
3. Generated reports
4. Screenshot/log showing full pipeline execution

Mini-Project 2: "The Process Monitor Dashboard"

Build a real-time process monitoring tool (like a mini- `htop`):

Requirements

1. Display a table of running processes with: PID, PPID, State, CPU%, MEM%, Command
2. Read data from `/proc/[pid]/stat` and `/proc/[pid]/status`
3. Refresh every 1 second using terminal control codes (ANSI escape sequences)
4. Support sorting by CPU, Memory, or PID (toggle with keyboard)
5. Support killing a selected process with a keystroke
6. Use threads: one for data collection, one for display, one for input

Project Structure

```
project_process_monitor/
├── Makefile
├── src/
│   ├── main.c            ← Entry point, terminal setup
│   ├── proc_reader.c     ← Read /proc filesystem
│   ├── display.c         ← ANSI-based terminal display
│   ├── input.c           ← Non-blocking keyboard input
│   └── proc_reader.h
├── test/
│   └── test_proc_reader.c
└── README.md
```

Deliverables

1. Working binary that displays live process info
2. Thread-safe data sharing between reader and display
3. Signal handling for clean exit (`Ctrl+C`)

Mini-Project 3: "The Library Plugin System"

Build a dynamic plugin architecture:

Requirements

1. A **host application** that loads `.so` plugins at runtime via `dlopen()`
2. Each plugin implements a standard interface:

```c
struct plugin_info {
    const char *name;
    const char *version;
    int (*init)(void);
    int (*execute)(const char *input, char *output, size_t out_len);
    void (*cleanup)(void);
};
```

3. Create at least 3 plugins:
    - **ROT13 plugin** — encodes/decodes text with ROT13
    - **Base64 plugin** — encodes/decodes Base64
    - **Hash plugin** — computes SHA-256 hash
4. Host scans a `plugins/` directory, loads all `.so` files
5. User can select a plugin and process data interactively
6. Build both static and shared library versions; compare sizes

Project Structure

```
project_plugin_system/
├── Makefile
├── include/
│   └── plugin_api.h        ← Plugin interface definition
├── src/
│   └── host.c              ← Plugin host application
├── plugins/
│   ├── rot13_plugin.c
│   ├── base64_plugin.c
│   └── hash_plugin.c
└── test/
    └── test_plugins.c
```

Mini-Project 4: "The IPC Message Broker"

Build a multi-process message broker that routes messages between producers and consumers:

Requirements

1. A **broker process** that accepts connections via Unix domain sockets
2. **Producer clients** send messages tagged with a topic (e.g., "sensor", "log", "alert")
3. **Consumer clients** subscribe to topics and receive matching messages
4. The broker uses **shared memory** for a high-speed message ring buffer
5. **POSIX semaphores** synchronize access to the ring buffer
6. **Message queues** as a fallback when shared memory is full
7. Proper signal handling for clean shutdown of all processes
8. A monitoring thread that reports throughput (messages/sec)

Project Structure

```
project_message_broker/
├── Makefile
├── include/
│   ├── broker.h
│   ├── message.h
│   └── ring_buffer.h
├── src/
│   ├── broker.c          ← Main broker process
│   ├── producer.c        ← Producer client
│   ├── consumer.c        ← Consumer client
│   └── ring_buffer.c     ← Shared memory ring buffer
└── test/
    └── test_broker.sh
```

Deliverables

1. Working broker, producer, and consumer binaries
2. Throughput benchmark results (messages/sec for each IPC method)
3. Clean shutdown when broker receives SIGTERM

Mini-Project 5: "The Real-Time Task Scheduler"

Build a user-space task scheduler that manages periodic and one-shot tasks:

Requirements

1. A **scheduler** that manages a list of tasks with priorities and periods
2. Tasks run as threads, each pinned to a specific CPU core
3. Support for:
   - Periodic tasks (e.g., run every 100ms)
   - One-shot tasks (run once after a delay)
   - Deadline-aware scheduling (warn if a task misses its deadline)
4. Uses `SCHED_FIFO` for real-time threads and `timerfd` for precise timing
5. A monitoring thread that tracks:
   - Jitter (deviation from expected period)
   - Deadline misses
   - CPU utilization per core
6. Graceful degradation: if a task exceeds its time budget, lower its priority

Project Structure

```
project_rt_scheduler/
├── Makefile
├── include/
│   ├── scheduler.h
│   └── task.h
├── src/
│   ├── scheduler.c       ← Main scheduler
│   ├── task_runner.c      ← Task execution engine
│   ├── monitor.c          ← Jitter/deadline monitor
│   └── main.c
└── test/
    └── test_scheduler.c
```

Deliverables

1. Working scheduler with at least 5 configured tasks
2. Jitter measurements with and without CPU isolation
3. Report showing deadline miss rates under different loads

Mini-Project 6: "The FUSE Encrypted Filesystem"

Build a FUSE filesystem that transparently encrypts all data:

Requirements

1. Mount a directory as an encrypted filesystem using FUSE
2. All files written are AES-256-CBC encrypted on disk
3. All files read are decrypted transparently
4. File names are also encrypted (using a deterministic scheme)
5. Key is derived from a passphrase using PBKDF2 or scrypt
6. Supports: read, write, create, delete, mkdir, rmdir, rename

7. Metadata (permissions, timestamps) is preserved
8. Secure temp file handling for intermediate operations

Project Structure

```
project_cryptfs/
├── Makefile
├── include/
│   ├── cryptfs.h
│   └── crypto_utils.h
├── src/
│   ├── cryptfs.c          ← FUSE operations
│   ├── crypto_utils.c     ← AES/key derivation
│   └── main.c             ← Mount/unmount logic
├── test/
│   ├── test_crypto.c
│   └── test_fs.sh
└── README.md
```

Deliverables

1. Working FUSE filesystem with encryption
2. Performance comparison: encrypted vs unencrypted I/O throughput
3. Security analysis: what's protected and what isn't

CAPSTONE PROJECT: "Multi-Process Secure File Vault with IPC & Real-Time Monitoring"

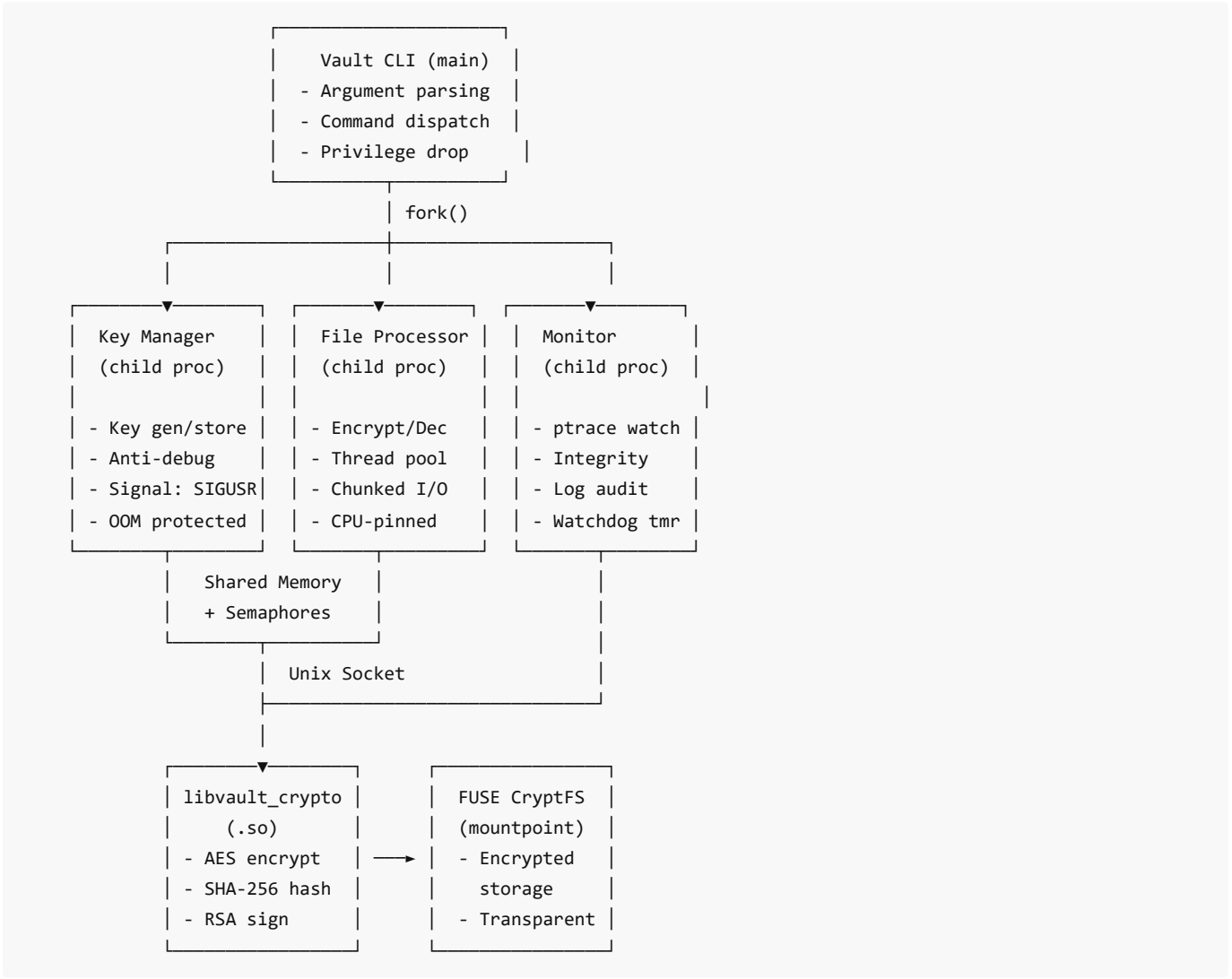
Build a complete system that combines ALL concepts from Part 1, Part 2, and Part 3:

Overview

A secure file vault application that:

- Encrypts/decrypts files using AES-256
- Manages multiple worker processes via `fork()`
- Uses threads for parallel encryption of large files (chunked)
- Traces its own processes with `ptrace` for integrity monitoring
- Implements anti-debugging protections for the key management module
- Packages crypto routines as a shared library (`libvault_crypto.so`)
- Logs all operations via proper error handling and structured logging
- Is profiled and optimized using PGO
- **Uses IPC (shared memory + message queues) for inter-process communication**
- **Pins encryption worker threads to dedicated CPU cores**
- **Uses POSIX timers for periodic integrity checks**
- **Manages temp files securely with `mkstemp()` / `O_TMPFILE`**
- **Drops root privileges after initial setup**
- **Stores encrypted files on a FUSE-mounted virtual filesystem**
- **Handles OOM gracefully with `oom_score_adj` protection for critical processes**

Architecture



Requirements

Phase 1: Library

- Create `libvault_crypto.so` with: AES encrypt/decrypt, SHA-256 hash, RSA sign/verify
- Build as both `.a` and `.so`; test independently

Phase 2: Process Architecture

- Main process forks three children: Key Manager, File Processor, Monitor
- IPC via pipes or shared memory for passing encryption keys
- Signal-based coordination (SIGUSR1 = key ready, SIGUSR2 = process file)
- Proper zombie reaping with `waitpid()`

Phase 3: Threaded File Processing

- Thread pool (4 workers) for parallel file chunk encryption
- Mutex-protected output file writing
- Condition variable for chunk queue

Phase 4: Security

- Anti-debugging on Key Manager process
- Integrity monitoring via Monitor process (ptrace-based)
- Proper error handling throughout with errno checking

Phase 5: IPC & Communication (Part 3)

- Shared memory ring buffer for passing file chunks between Key Manager and File Processor
- POSIX semaphores for ring buffer synchronization
- Unix domain socket between Monitor and all child processes for health reporting
- Message queue for audit log events

Phase 6: Real-Time & Scheduling (Part 3)

- Pin File Processor threads to specific CPU cores using `sched_setaffinity()`
- Set Key Manager to `SCHED_FIFO` with high priority
- Use `timerfd` for periodic integrity check (every 2 seconds)
- Measure and log jitter in the watchdog timer
- Protect Key Manager from OOM killer (`oom_score_adj = -900`)

Phase 7: System Hardening (Part 3)

- Drop root privileges after binding to privileged ports / opening key files
- Use `mkstemp()` for all temporary files; unlink immediately after opening
- Secure temp directory with `mkdtemp()` and `0700` permissions
- Set `PR_SET_DUMPABLE` to 0 to prevent core dumps of sensitive data

Phase 8: FUSE Integration (Part 3)

- Mount a FUSE filesystem as the encrypted storage backend
- All vault data is stored through the FUSE layer
- Transparent encryption/decryption at the filesystem level
- FAT-formatted disk image as the underlying storage

Phase 9: Build & Profile

- Complete Makefile with all stages
- PGO-optimized build
- Profile with gprof, compare before/after optimization
- ELF analysis report of final binaries

Project Structure

```
project_file_vault/
├── Makefile
├── include/
│   ├── vault_crypto.h
│   ├── vault_process.h
│   ├── vault_ipc.h
│   ├── vault_timer.h
│   └── vault_common.h
├── lib/
│   ├── vault_crypto.c
│   └── Makefile
├── src/
│   ├── main.c
│   ├── key_manager.c
│   ├── file_processor.c
│   ├── monitor.c
│   ├── thread_pool.c
│   ├── ipc_shm.c
│   ├── watchdog.c
│   └── priv_drop.c
├── fuse/
```

```
│   ├── vault_fs.c
│   └── Makefile
├── test/
│   ├── test_crypto.c
│   ├── test_threading.c
│   ├── test_ipc.c
│   ├── test_fuse.sh
│   └── run_tests.sh
├── profile/
│   └── profile_report.txt
└── docs/
    └── architecture.md
```

Deliverables

1. Complete source code with Makefile
2. Both static and shared library builds
3. Profile report comparing PGO vs non-PGO
4. ELF analysis report of all generated binaries
5. Test results showing all components working
6. Architecture diagram (as above or more detailed)
7. IPC throughput benchmark results
8. Jitter measurement report for the watchdog timer
9. Security audit checklist (privilege dropping, temp files, OOM protection)

End of Linux System Programming — Hands-On Lab Manual